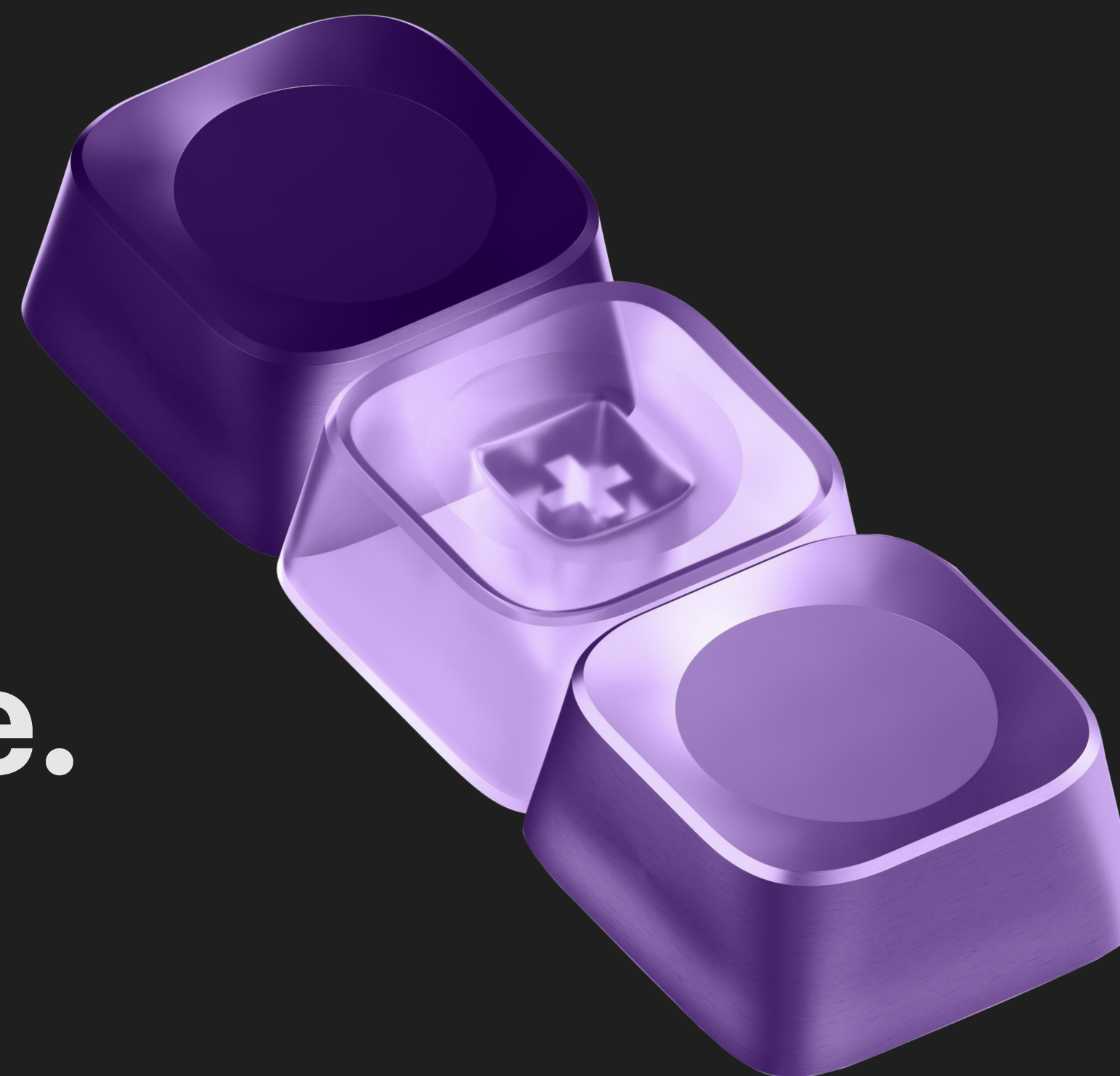


Event-driven architecture. Работа с Kafka

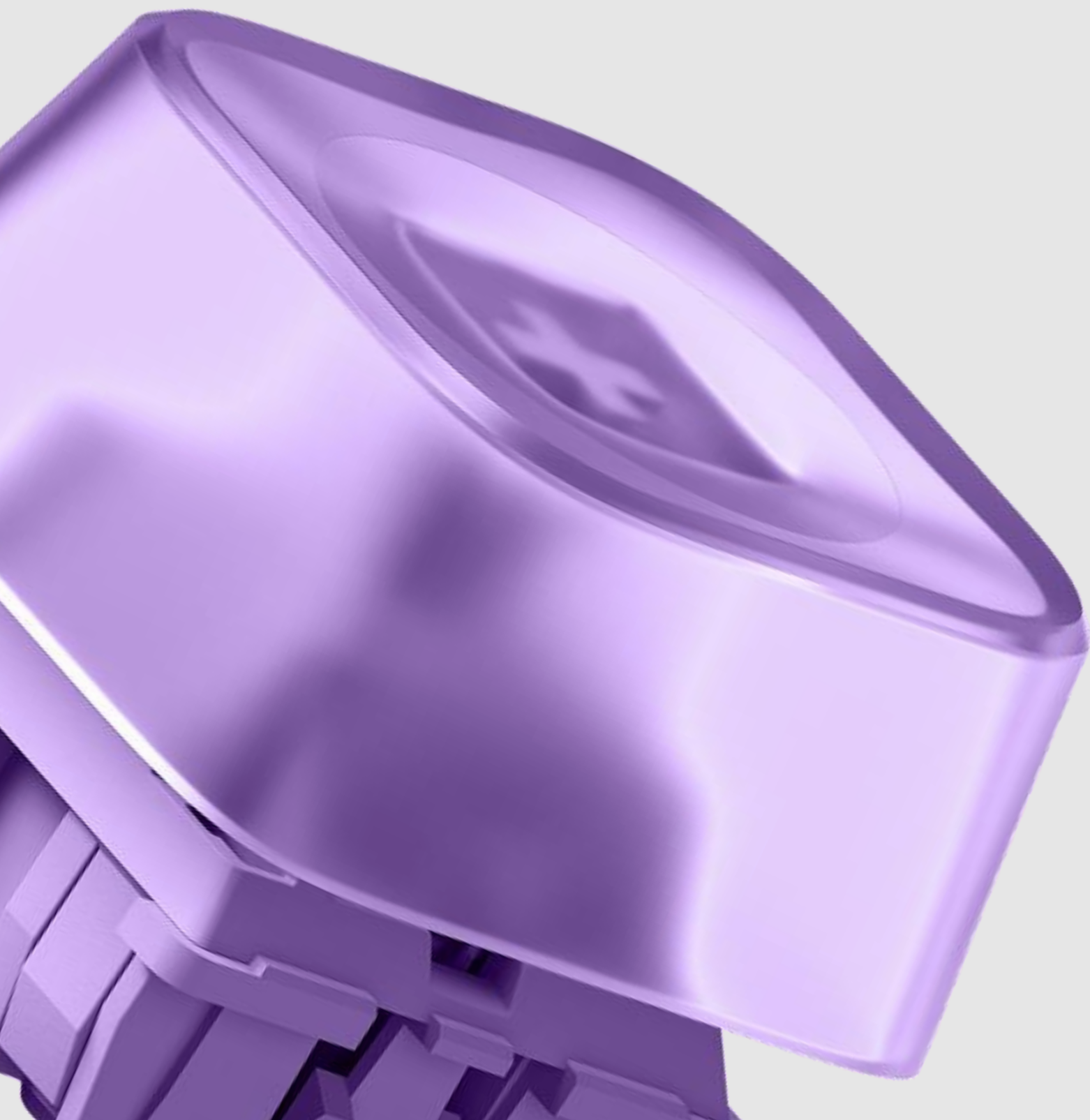
JAVA SPRING

НЕДЕЛЯ 14



План семинара

- Консьюмер группа. Ребалансировки и их политики
- DLT (DLQ) & Retry
- Kafka semantics & transaction

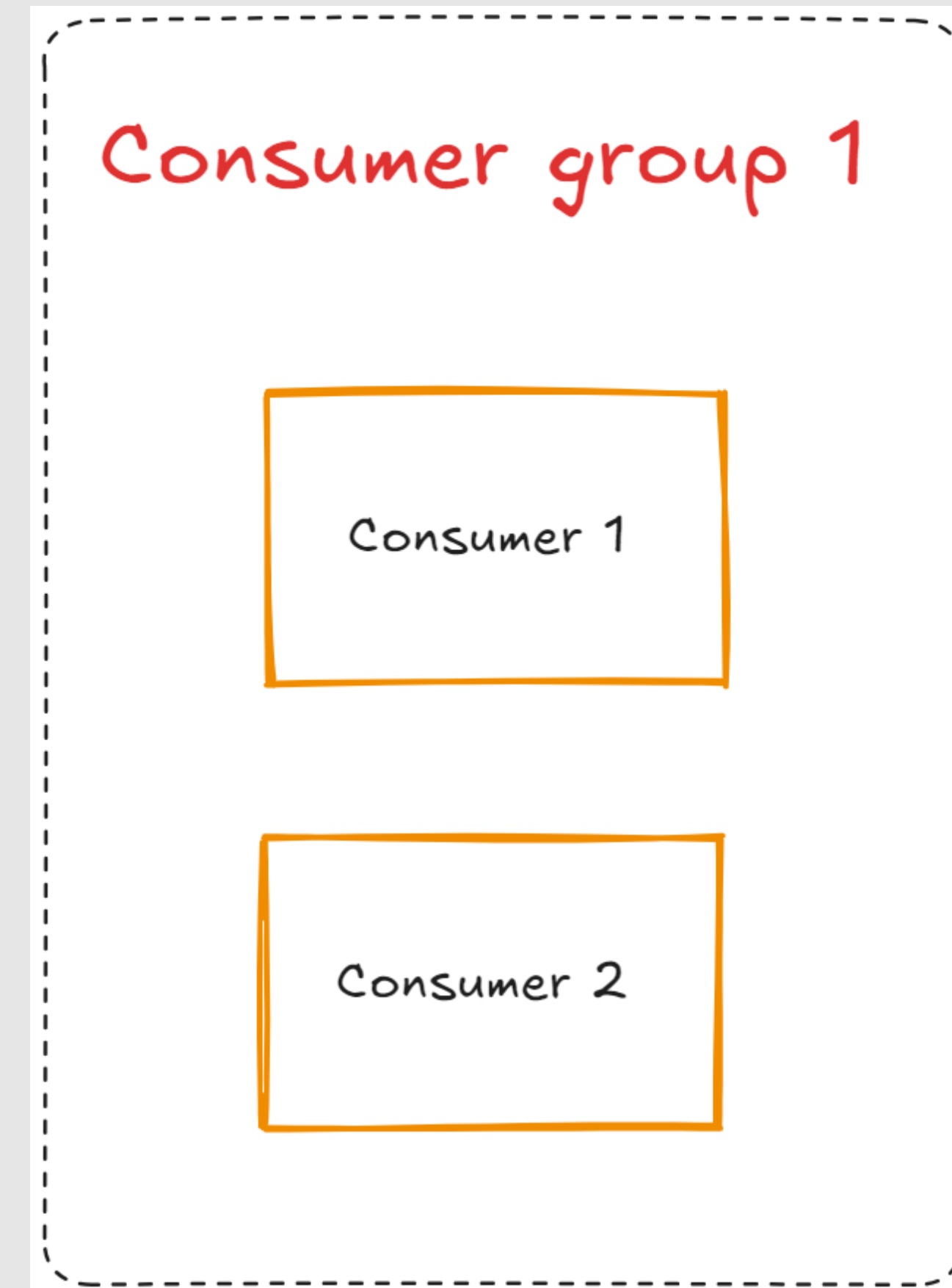


Консьюмер группа. Ребалансировки и их политики



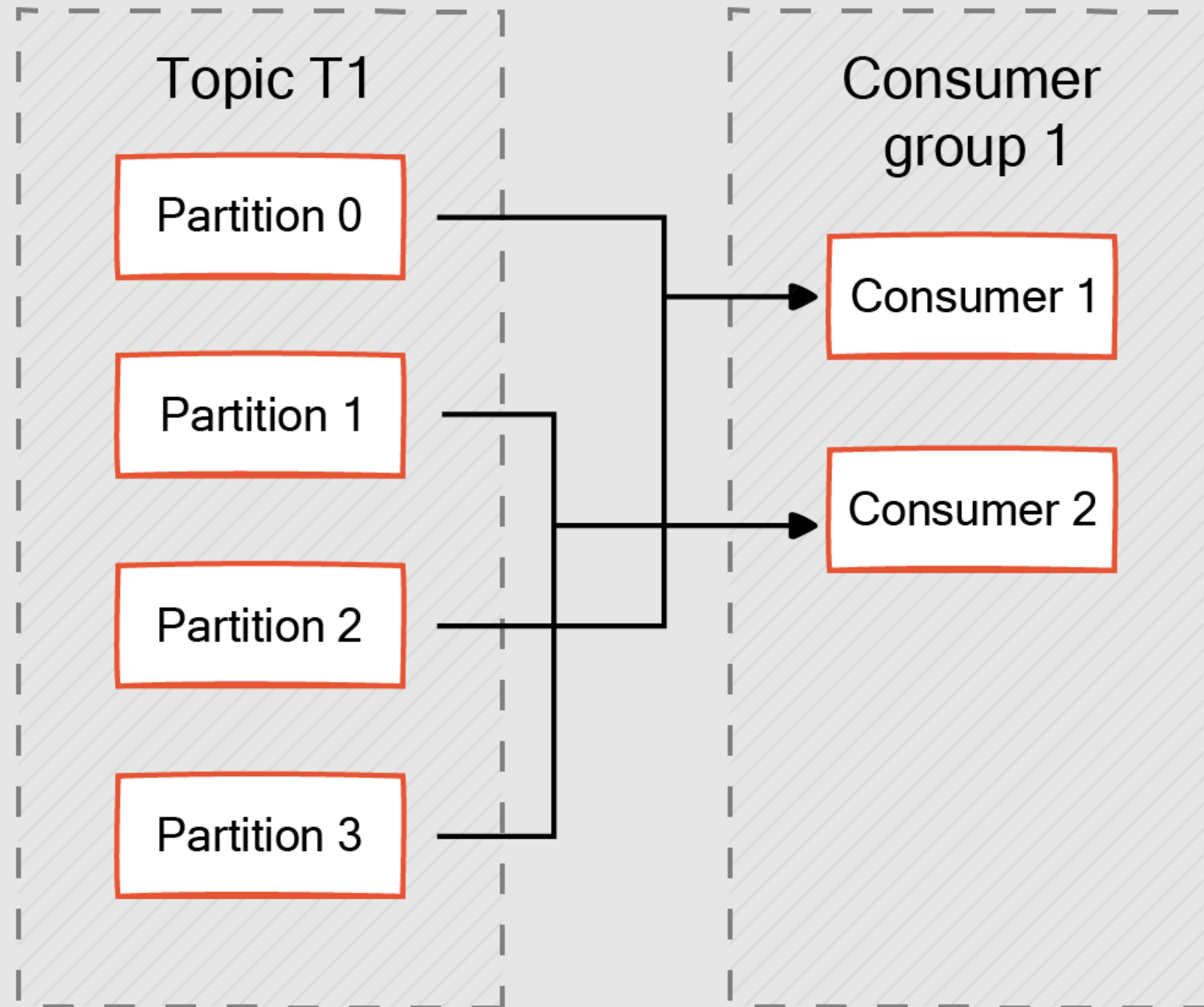
Consumer group

- **Kafka consumer group** – consumers могут работать в составе consumer group, где один или несколько потребителей consumers читают с одного и того же топика.
- **Single membership** – в consumer group может быть только один Kafka consumer.
- **Working mechanism** - члены одной и той же consumer group получают сообщения из разных topic partitions
- **Self-balancing** - Каждому consumer в consumer group назначается отдельный partition. Это может привести к простоям пользователей или к появлению нескольких разделов для каждого потребителя.
- **Multiple consumer groups** - consumers из двух разных consumer groups могут быть присвоены к одному и тому же partition.
- **Group coordinator** - координатор группы облегчает partition assignment и отслеживает чтение сообщений.
- **Partition assignment strategy** – Range, Round-robin, sticky assignment стратегии используются для назначения партиций



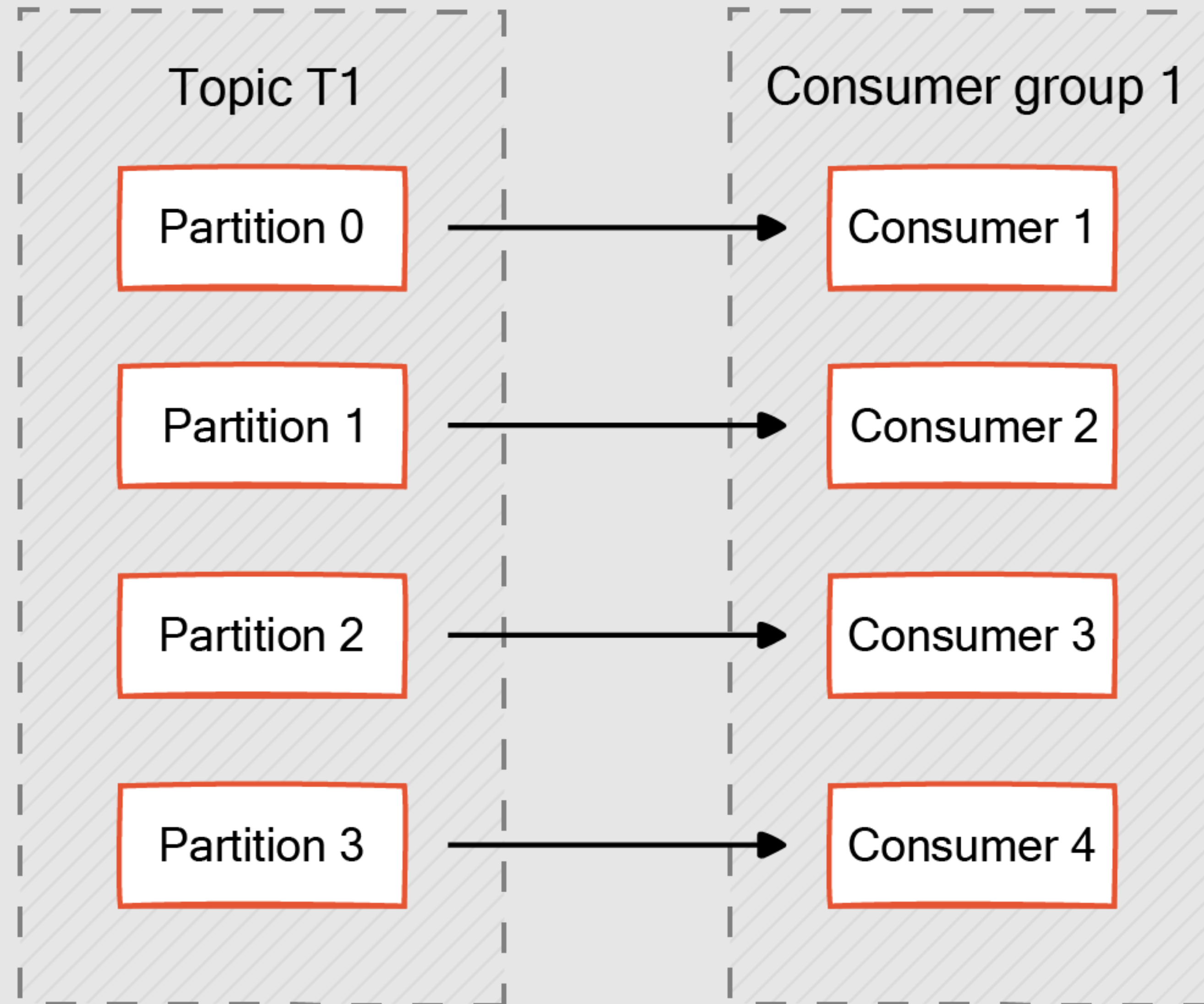
Consumer group

Partitions больше чем consumers



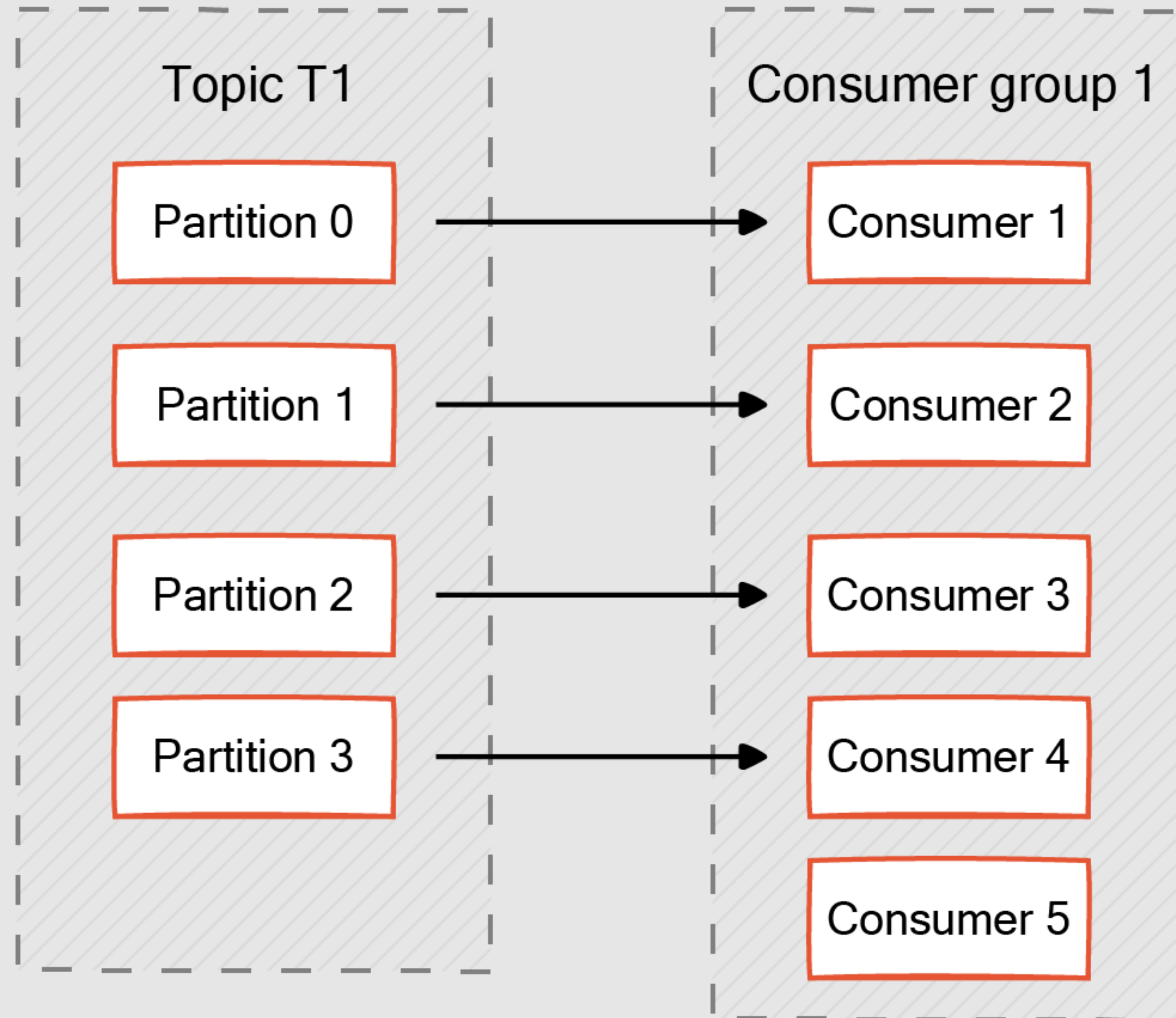
Consumer group

Partitions = consumers



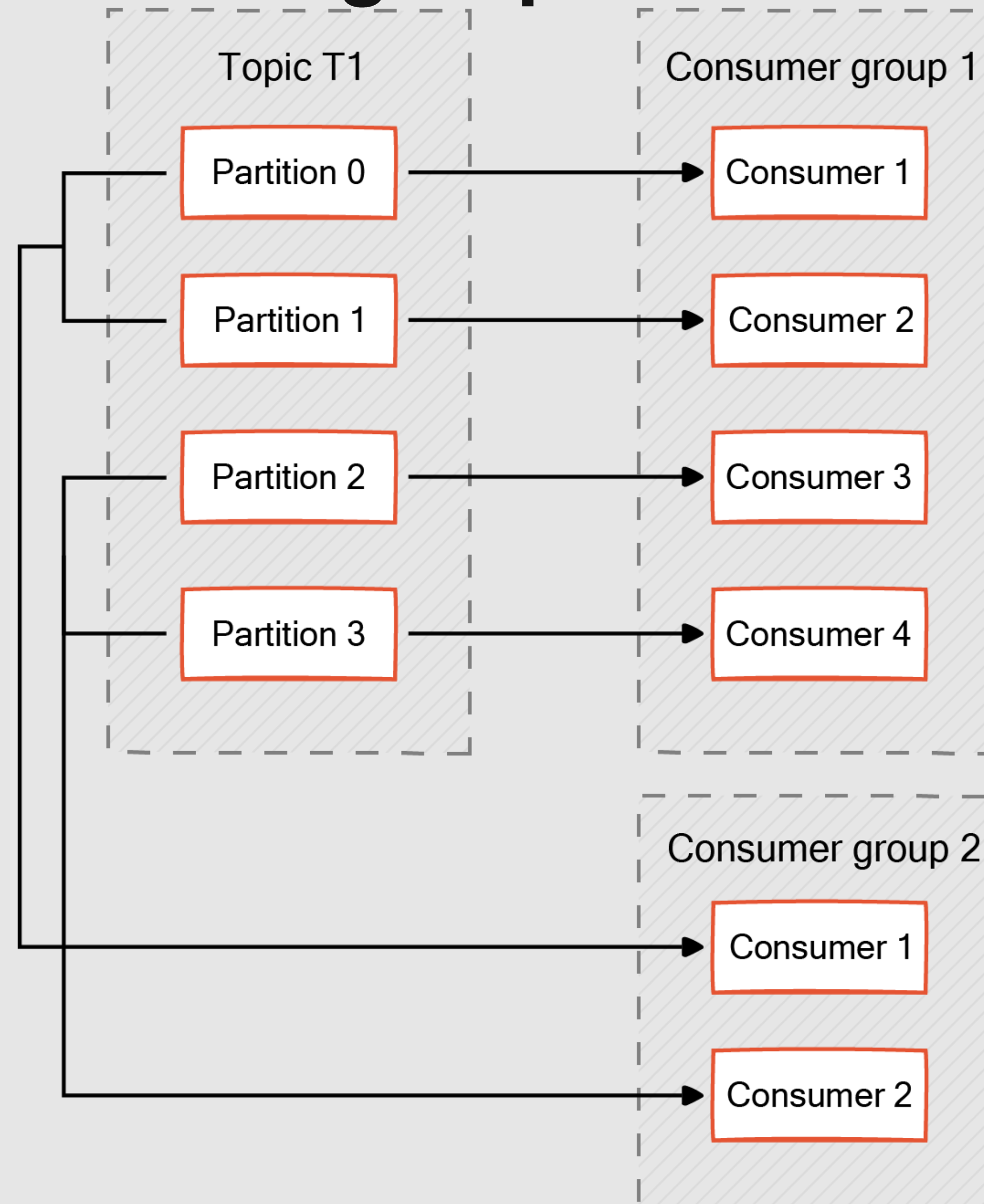
Consumer group

Partitions меньше чем consumers



Consumer group

Множественные consumer groups

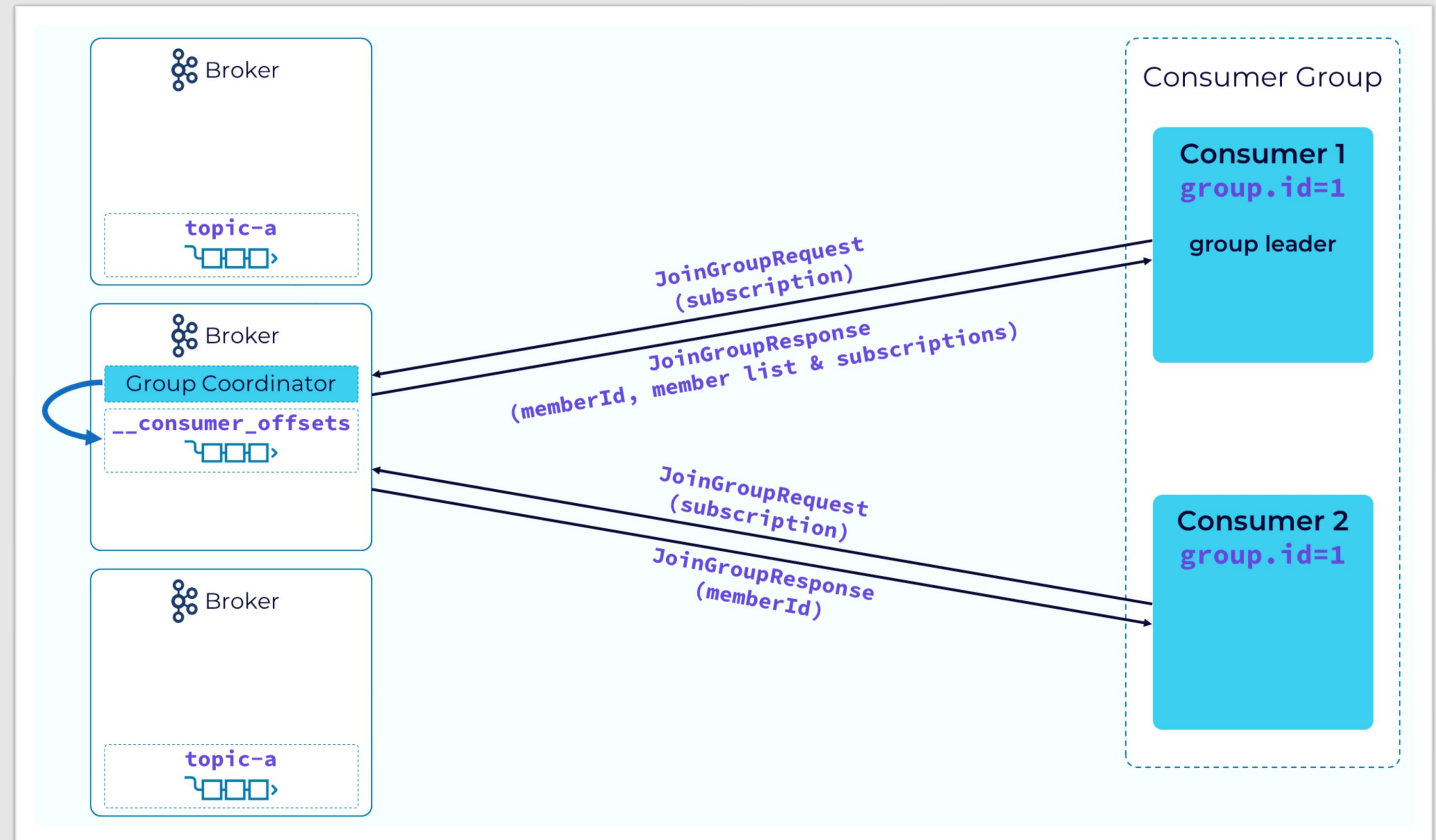


Consumer group

Назначение партиций (Partitions assigned)

Group coordinator:

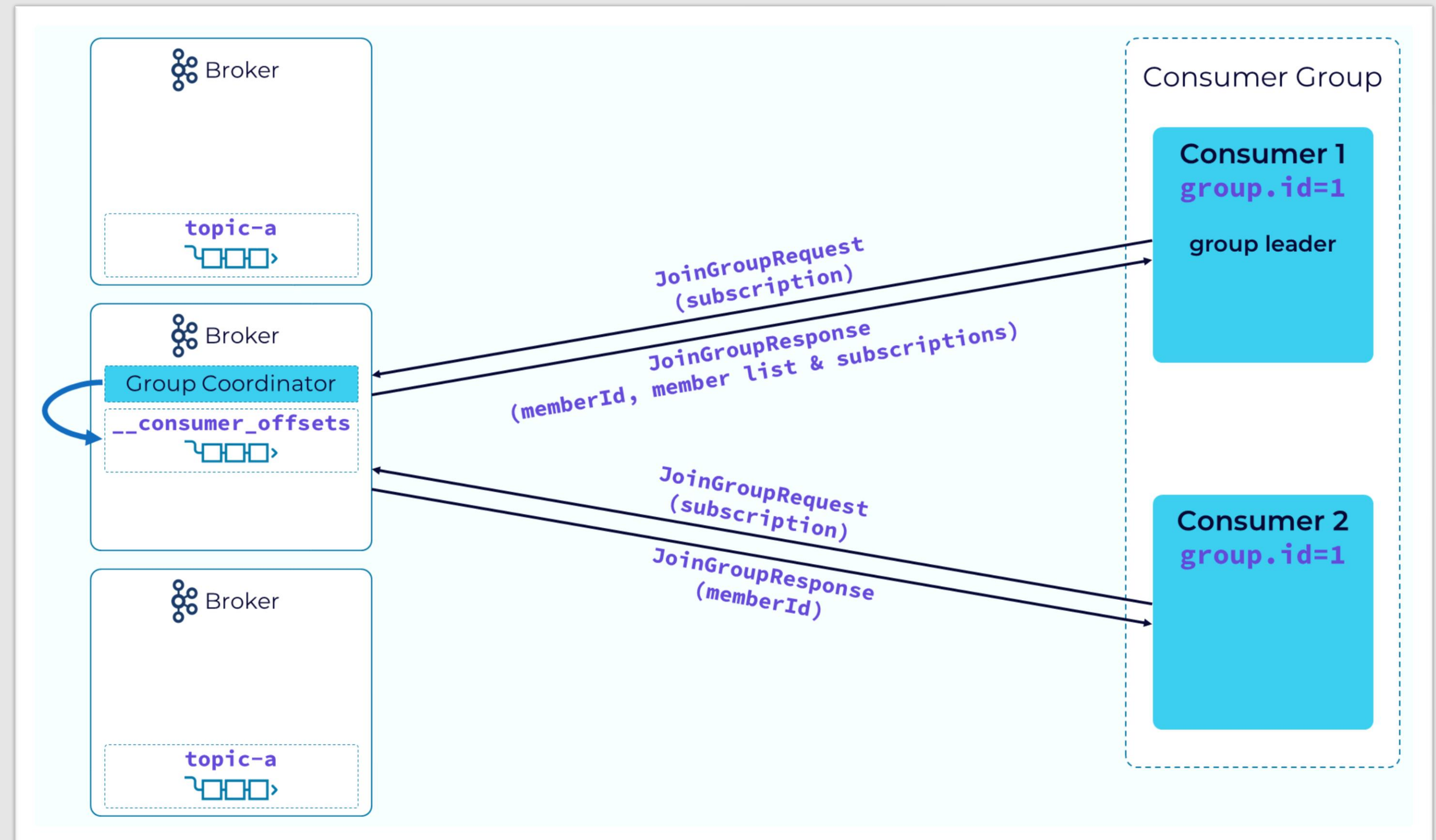
- Координатор группы способствует равномерному распределению partitions между членами consumer group даже при изменении членства в группе.
- Координатор группы является частью Kafka broker или Kafka server.
- Назначение partition достигается за счет использования internal topic Kafka для поддержки и управления метаданными группы.



Consumer group

Group coordinator functions

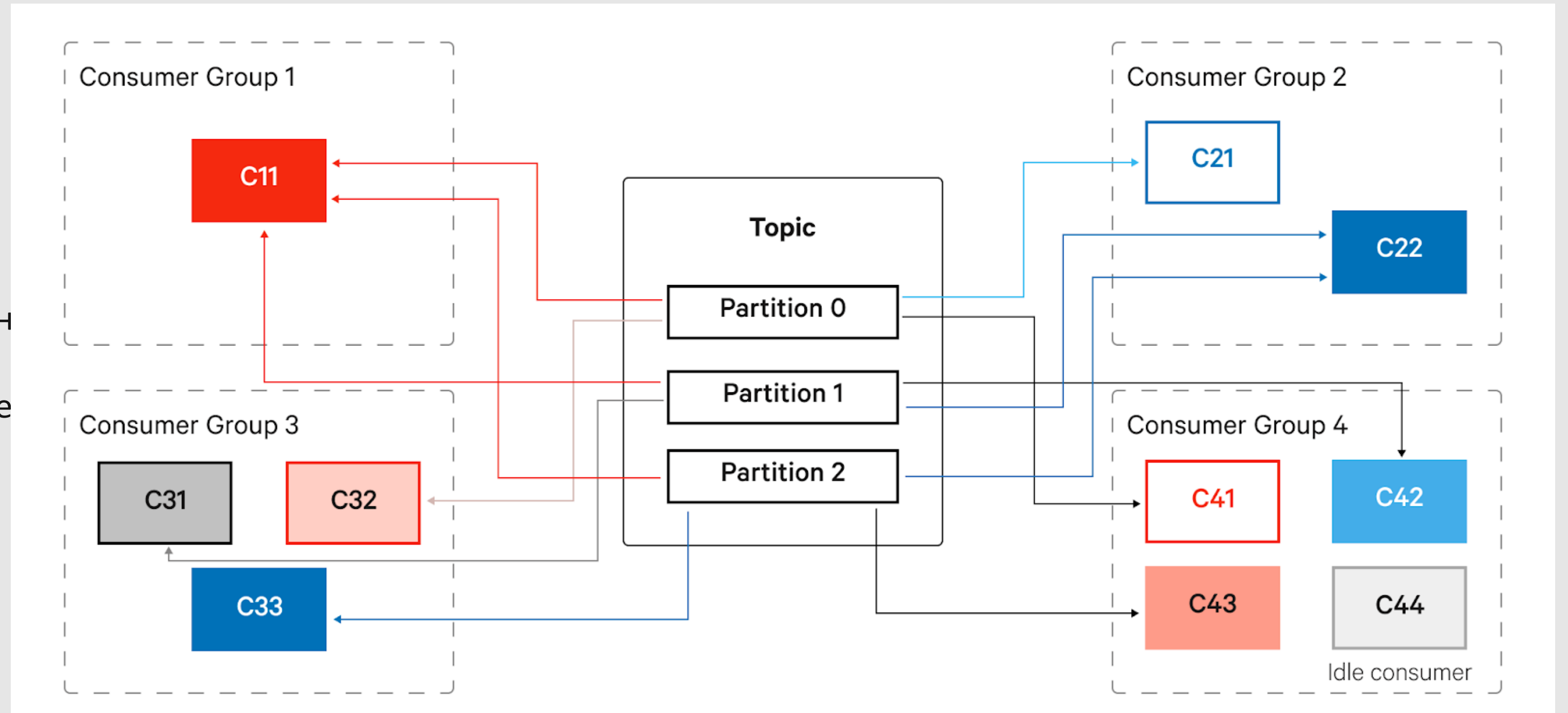
- **Assigning partitions** - когда consumer присоединяется к группе или покидает ее, координатор группы перераспределяет партии между оставшимися consumers, используя алгоритм перебалансировки.
- **Heartbeat mechanism** - координатор группы отслеживает состояние consumers, получая от них периодические сигналы, называемые *heartbeats*. Пока потребитель регулярно отправляет сообщения *heartbeats*, предполагается, что он подключен к сети.
- **Tracking message consumption** - Координатор группы отслеживает сообщения, которые приняла consumer за счет consumer offset.



Consumer group

Partition assignment strategy

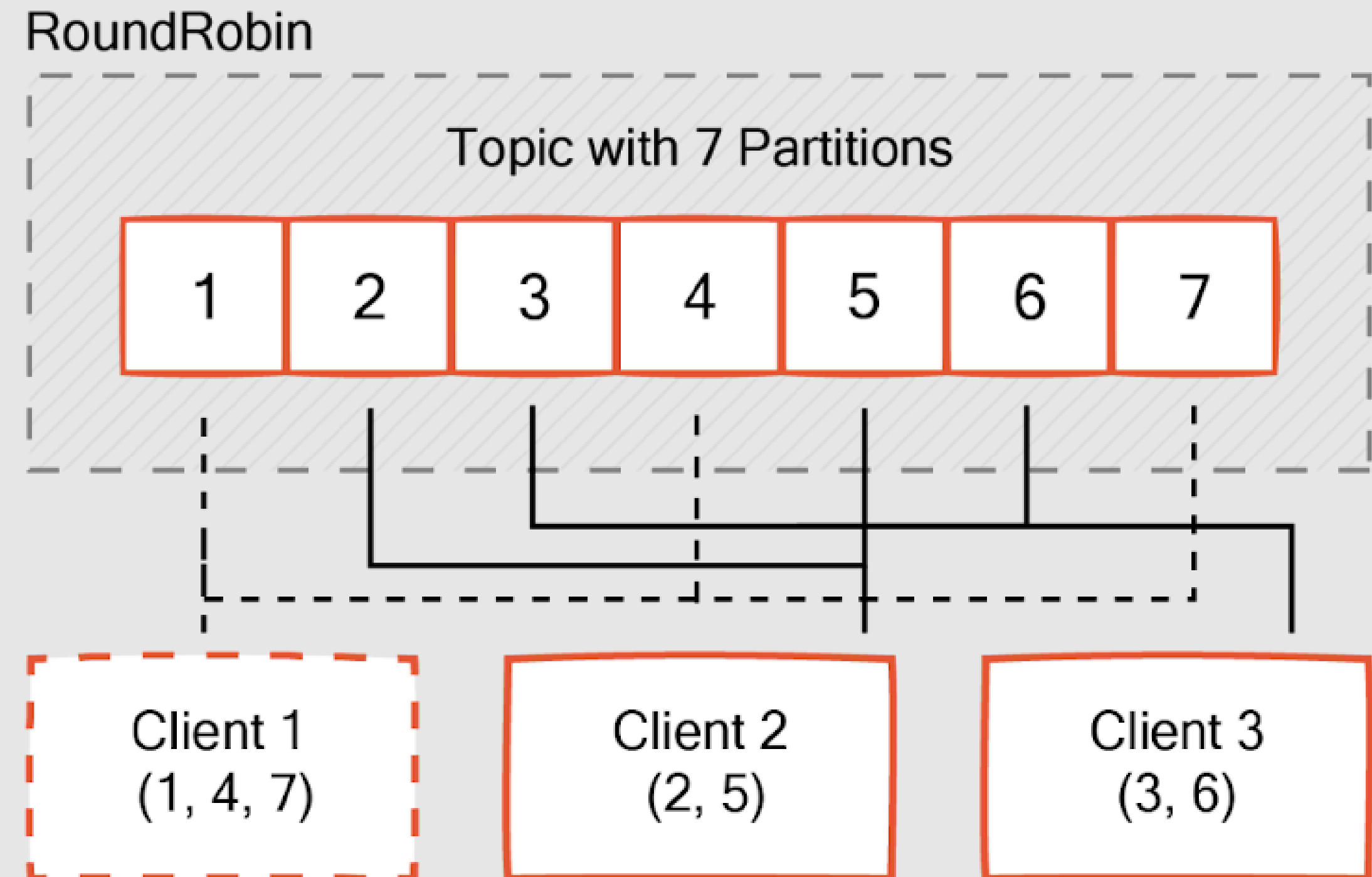
- Координатор группы использует partition assignment strategy для назначения партиций в consumer group.
- Класс PartitionAssignor выступает в качестве лица, принимающего решения в архитектуре Kafka.
- Он принимает информацию о consumers и подписанных топиках, а затем выводит сопоставление, которое назначает определенные partitions (сегменты сообщения) каждому consumer в группе.



Consumer group

Round robin assignor

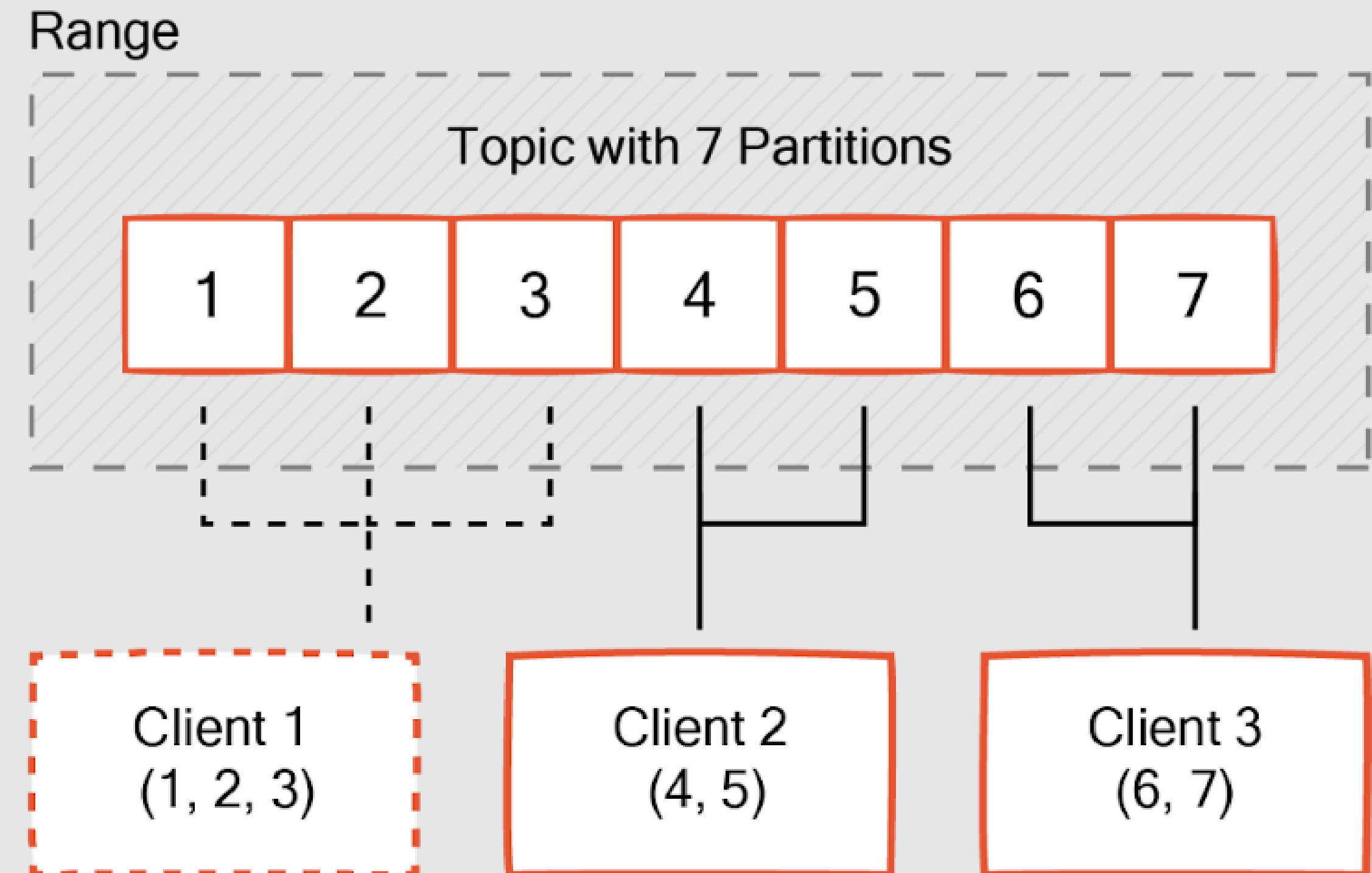
- При round robin все партии из всех подписанных топиков берутся и назначаются пользователям последовательно, по одному за раз.
- Это приводит к уменьшению числа незанятых consumers.
- Используется по умолчанию



Consumer group

Range assignor

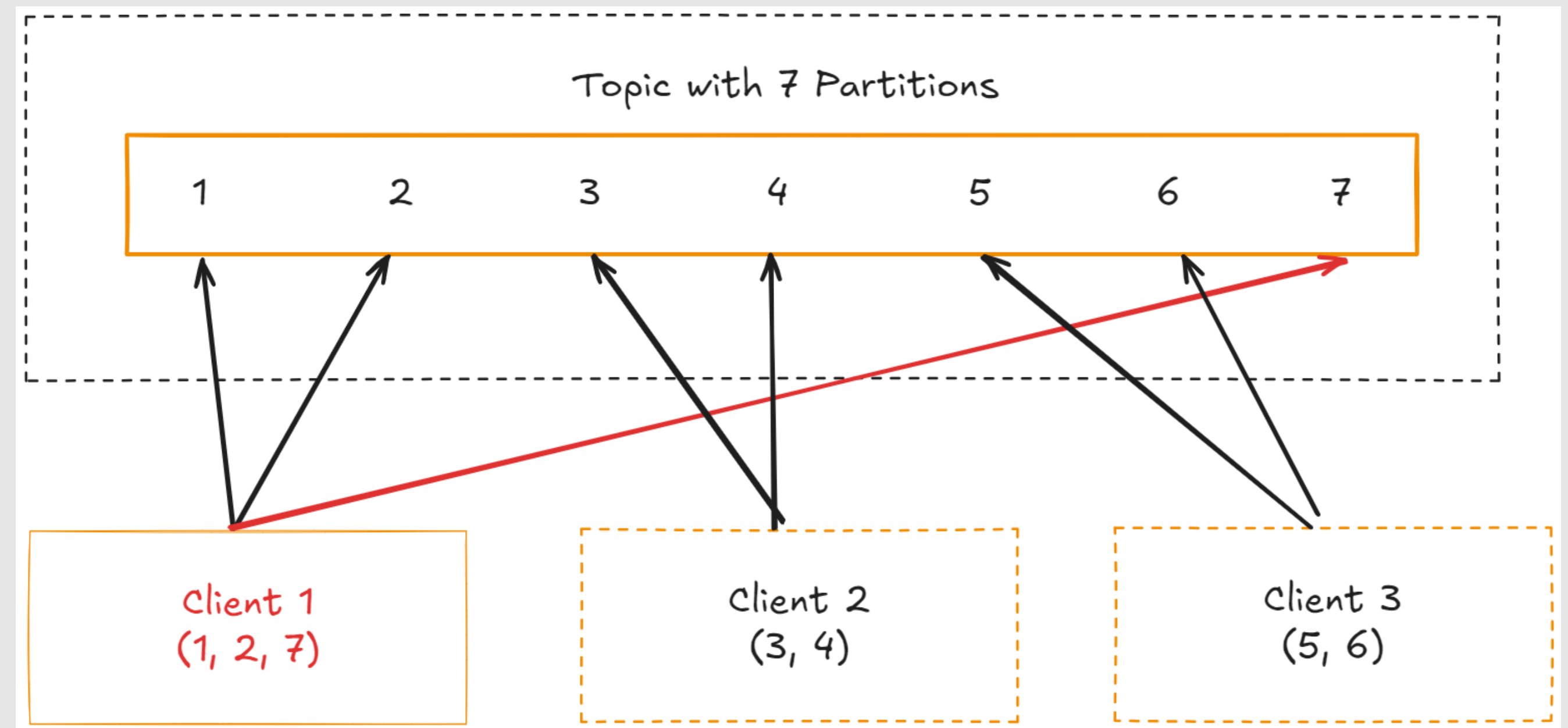
- При Range каждому потребителю выделяются последовательные партии из каждого топика, на которую он подписан.
- Это означает, что первый раздел назначается первому потребителю, второй - второму потребителю и так далее.
- Неравномерное количество разделов в каждом топике может привести к простоям пользователей.



Consumer group

Sticky assignor

- Это работает почти так же, как round-robin assignor, но имеет дополнительное преимущество, поскольку при переназначении partitions сохраняется как можно больше существующих assignments.
- Это помогает сэкономить часть накладных расходов на обработку, когда topic partitions перемещаются от одного consumer к другому во время ребалансировки.



Consumer group

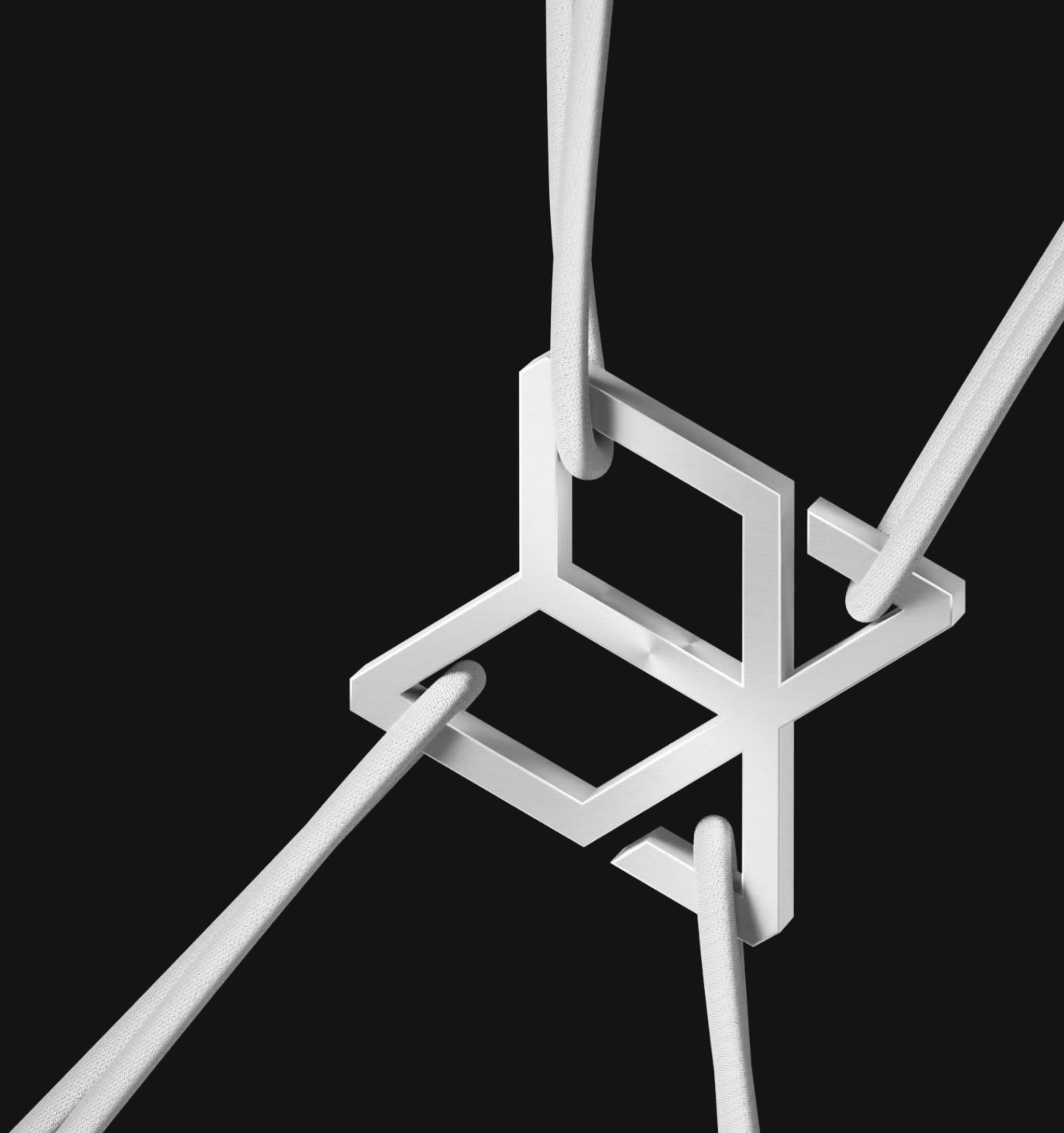
Consumer rebalancing

- Время от времени consumers могут добавляться в consumer group или удаляться из нее. При добавлении нового consumer он начинает получать сообщения из партиций, ранее назначенных другому consumer. Аналогично, когда пользователь удаляется, его партиции назначаются другому оставшемуся активному consumer. Такое переназначение на партиции от одного consumer другому называется перебалансировкой.
- Перебалансировка имеет высокое значение, поскольку обеспечивает высокую доступность и масштабируемость, позволяя пользователям безопасно добавлять и удалять consumers

Существует 2 вида протоколов ребалансировки:

- **Eager rebalancing** - все consumers прекращают читать, отвязываются от своих партиций, пересоединяются к группе (rejoin the group), а затем получают новые партиции, назначенные им. Это приводит к небольшому перерыву в работе (consumer unavailability), когда во всей группе нет consumers (**Round robin, Range, Sticky assigners**)
- **Cooperative rebalancing** (incremental rebalancing) - стратегия выполняет перебалансировку в несколько этапов. Она включает в себя переназначение небольшого подмножества партиций от одного consumer до другого, что позволяет пользователям продолжать обрабатывать сообщения от партиций, которые не переназначены, и избежать полной недоступности (**CooperativeStickyAssignor**)

Kafka Producer Partitioner



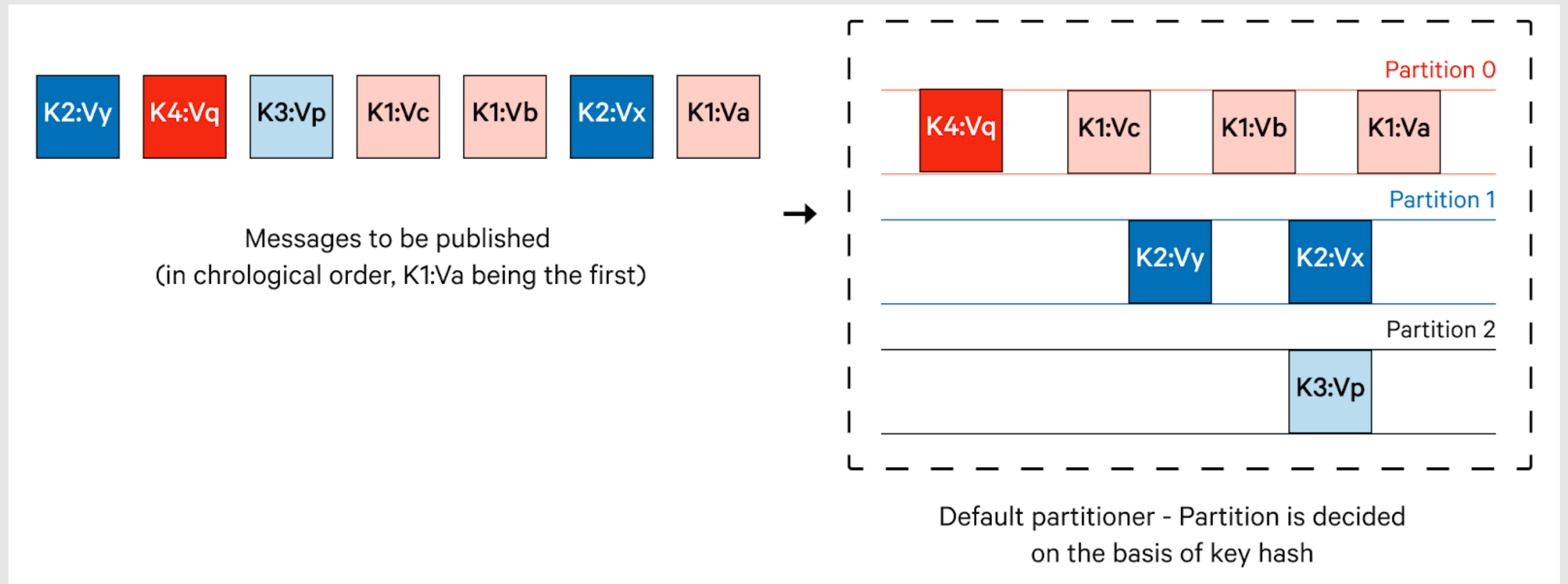
Kafka Producer Partitioning strategies

- Default
- Round robin
- Uniform sticky
- Custom

Kafka Producer

Default partitioner

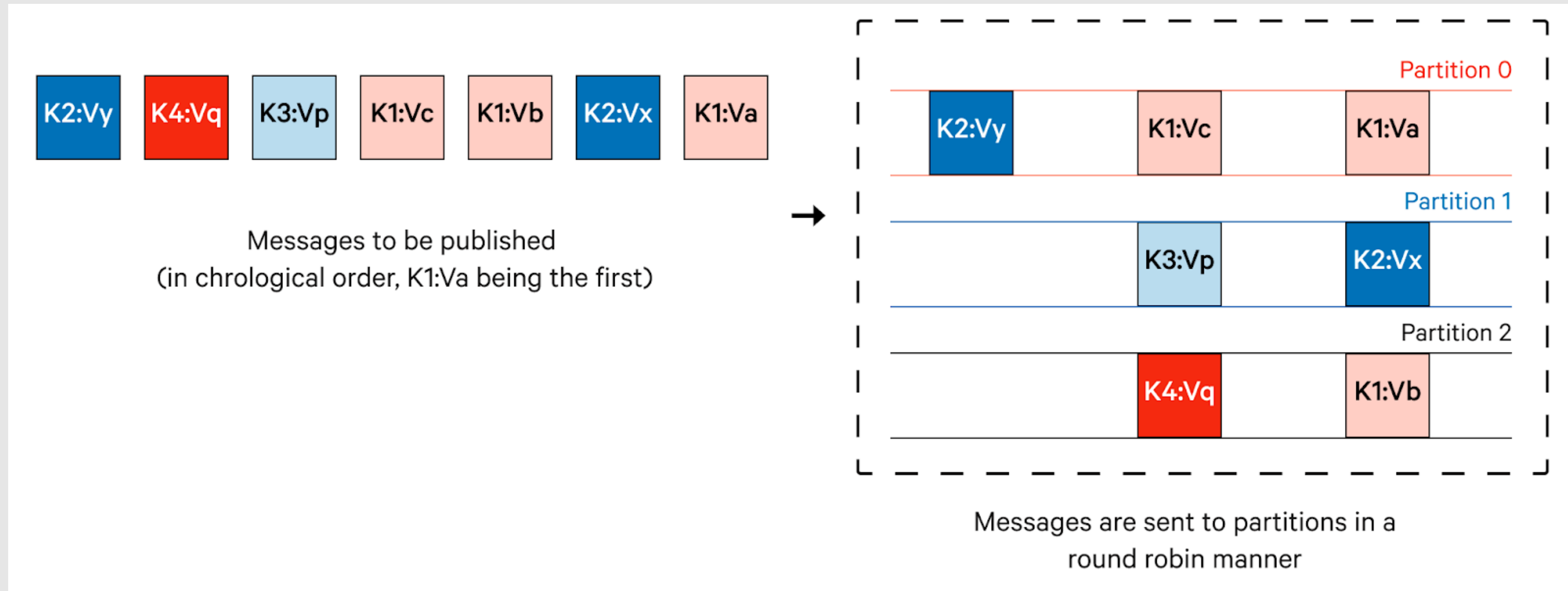
- Используется по умолчанию
- Если значение ключа null – то запись отправляется случайным образом
- Если ключ существует, то Kafka хэширует его, и результат используется для привязки сообщения к определенной партии



Kafka Producer

Round robin partitioner

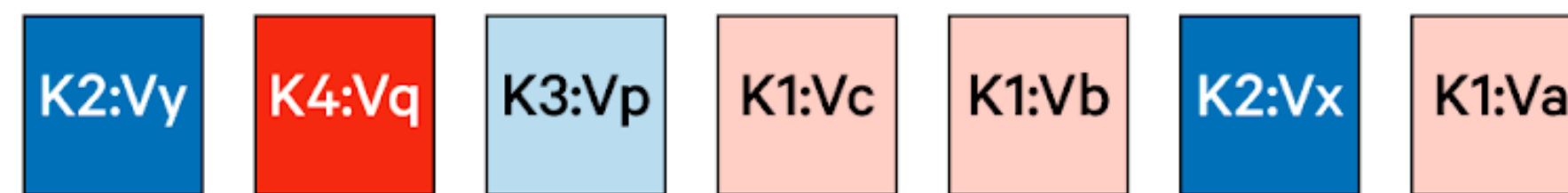
- Равномерное распределение сообщений между партициями
- Отсутствует гарантия последовательности отправки сообщений между партициями



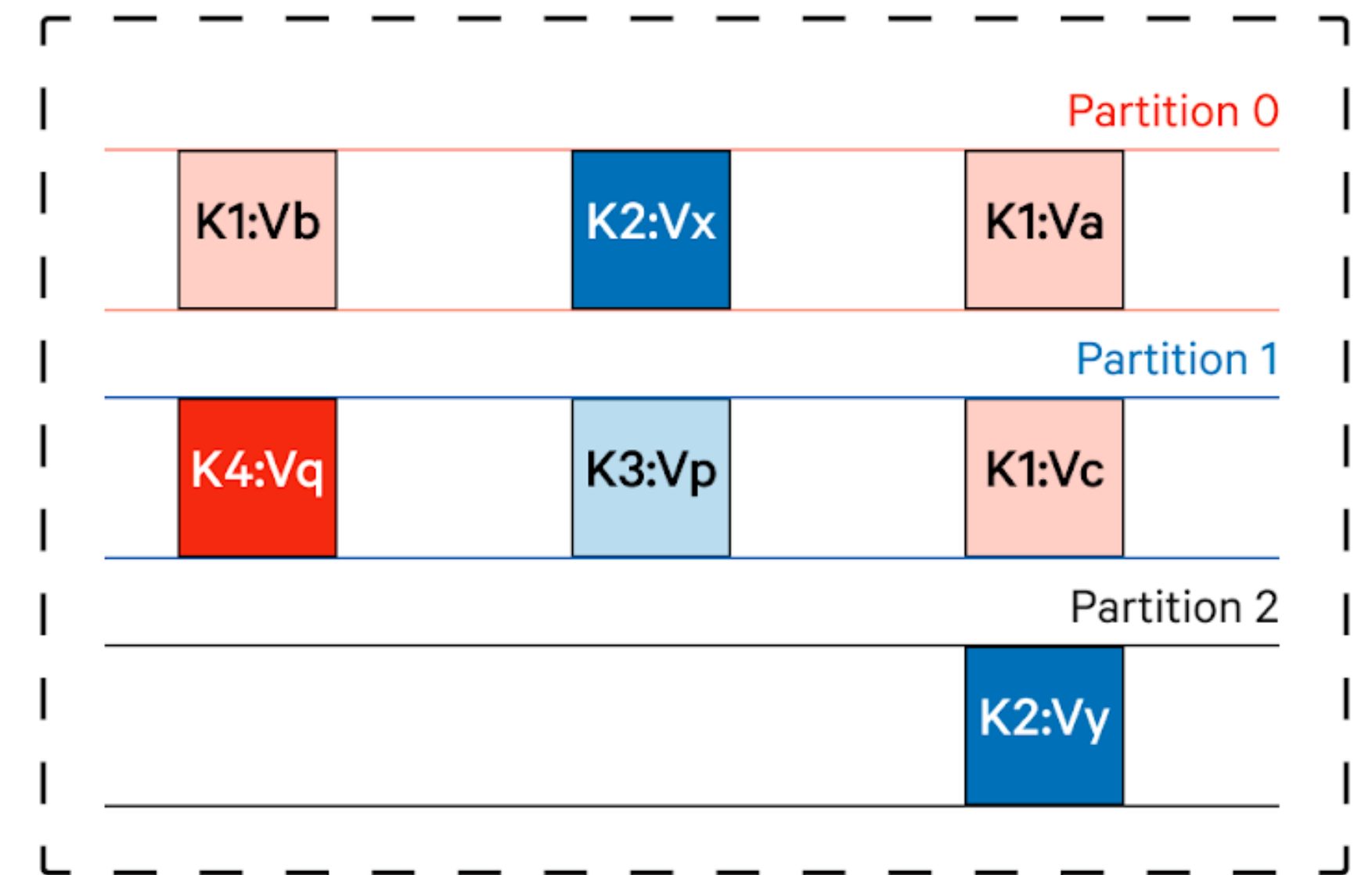
Kafka Producer

Uniform sticky partitioner

- Призван решить проблему частой отправки сообщений
- Является по своей сути батч запросом (хотя батч можно включить и без sticky partitioner)



Messages to be published
(in chronological order, K1:Va being the first)



Sticky partition changes only when batch is full. Also notice that records with same key are not guaranteed to be in the same partition

Kafka Producer Custom partitioner

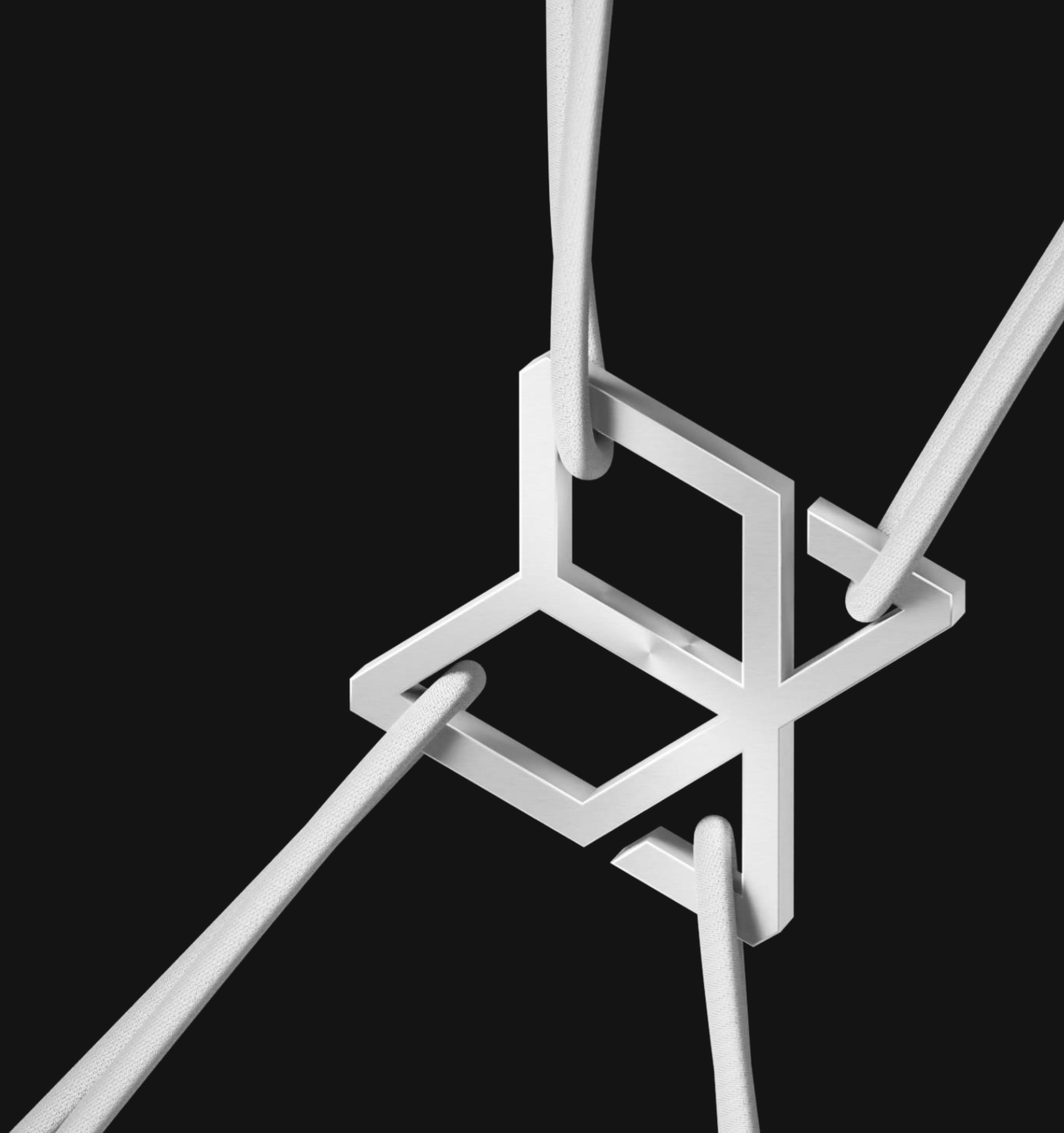
- Подходит для специфичных кейсов
- Необходимо реализовать интерфейс **Partitioner**

Kafka Producer

Custom partitioner

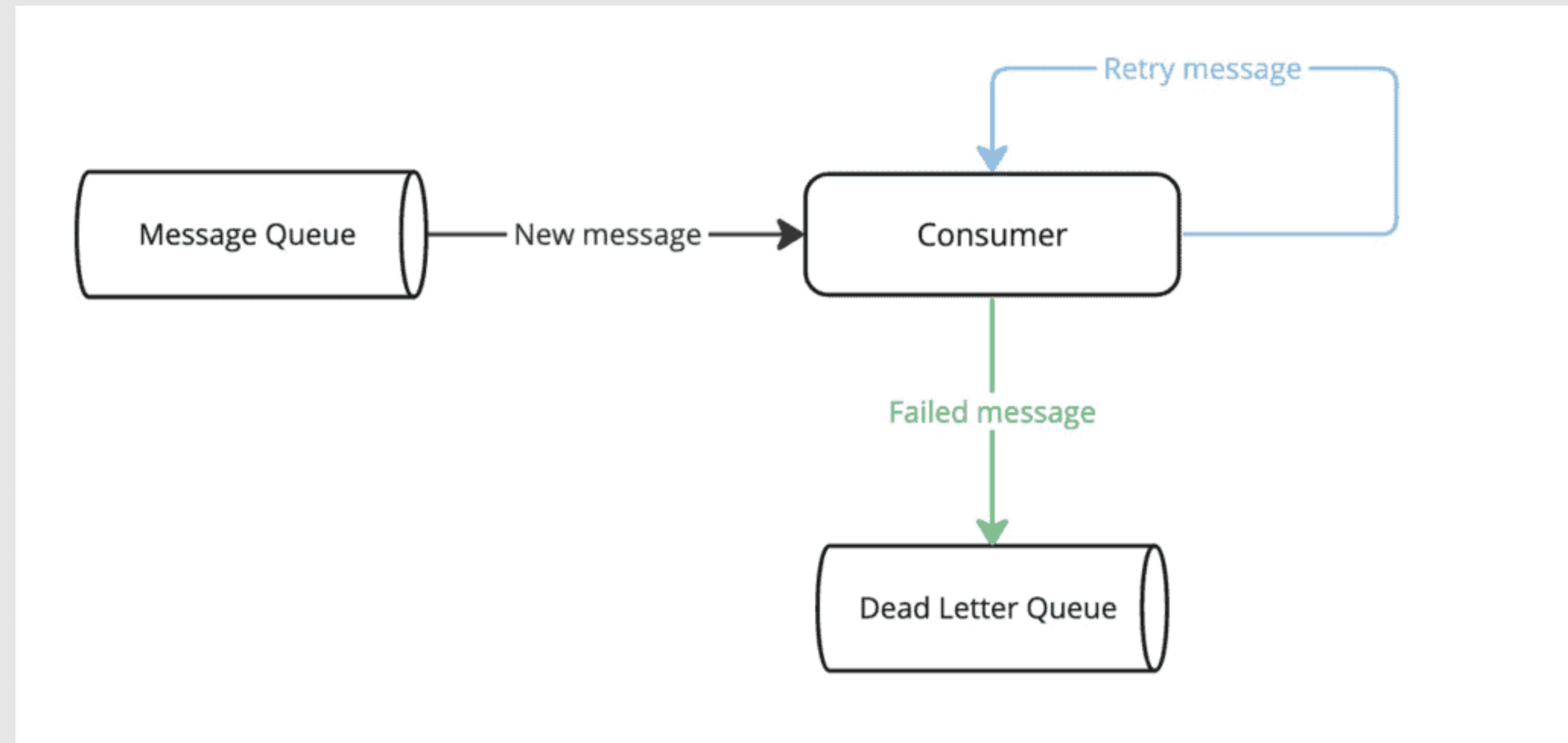
```
• public class PowerUserPartitioner implements Partitioner {  
  
    public int partition(String topic, Object key, byte[] keyBytes,  
                        Object value, byte[] valueBytes, Cluster cluster) {  
  
        List<PartitionInfo> partitions = cluster.partitionsForTopic(topic);  
        int numPartitions = partitions.size();  
        if ((keyBytes == null) || (!(key instanceof String)))  
            throw new InvalidRecordException("Record must have a valid string key");  
  
        if (((String) key).equals("CEO"))  
            return numPartitions - 1; // Messages with key "CEO" will always go to the last partition  
  
        // Other records will get hashed to the rest of the partitions  
        return Math.abs(Utils.murmur2(keyBytes)) % (numPartitions - 1);  
    }  
  
    public void configure(Map<String, ?> configs) {}  
  
    public void close() {}  
}
```

DLT & Retry



DLT (DLQ)

→ **Dead Letter Queue** используется для хранения сообщений, которые не могут быть корректно обработаны



DLT

Drawbacks

- Не рекомендуется использовать DLQ для очереди, где важен точный порядок сообщений
- Стоит провести анализ об использовании при огромном потоке аналитических сообщений
- Стоит провести анализ об использовании, когда сообщения быстро устаревают

DLT

Retries

- Обычно принято повторять обработку события в случае ошибок, прежде чем отправлять его в DLT.
- Ретраи бывают как блокирующие, так и неблокирующие
- В случае DLT используется механизм неблокирующих ретраев, предоставляемый Spring Kafka.
- Требуются точные протестированные настройки – иначе сервис может уйти в **”бесконечную”** повторную обработку!!!

DLT

Consumer Acknowledge

Режим подтверждения определяет, когда брокер обновляет offset.
Можно выделить:

- After-interval
- After-processing
- Manual

DLT

Ack Mode

- RECORD – коммит оффсета после каждого обработанного сообщения
- BATCH - коммит оффсета после обработки всех сообщений вызванного в `poll()`
- TIME - коммит оффсета после обработки всех сообщений вызванного в `poll()`, но не чаще установленного времени `ackTime`
- COUNT - коммит оффсета после обработки всех сообщений вызванного в `poll()`, но не чаще установленного количества `ackCount`
- COUNT_TIME – срабатывает либо по COUNT, либо по TIME
- MANUAL – коммитит offset, когда вызывается вручную метод `Acknowledge.acknowledge()` но не сразу, а по принципу, как и у BATCH.
- MANUAL_IMMEDIATE - коммитит offset немедленно, когда вызывается вручную метод `Acknowledge.acknowledge()`.

⚠ IMPORTANT

You can set the `AckMode` mode you prefer, but `RECORD` is suggested.

Retry

Blocking retry

@Bean

```
public CommonErrorHandler errorHandler() {  
    BackOff fixedBackOff = new FixedBackOff(interval, maxAttempts);  
    DefaultErrorHandler errorHandler = new DefaultErrorHandler((consumerRecord, exception) -> {  
        // logic to execute when all the retry attempts are exhausted  
    }, fixedBackOff);  
  
    errorHandler.addRetryableExceptions(TimeoutException.class);  
    errorHandler.addNotRetryableExceptions(DeserializationException.class, NullPointerException.class);  
  
    return errorHandler;  
}
```

Retry

Blocking retry

BackOff имлментации:

- FixedBackOff (interval, maxAttempts)
- ExponentialBackOff (initialInterval, multiplier)
- ExponentialBackOffWithMaxRetries (maxRetries)
- MyCustomBackOff (...)

Retry

Blocking retry

```
@Bean
public KafkaListenerContainerFactory<ConcurrentMessageListenerContainer<String, Object >> retryKafkaListenerContainerFactory(
    @Qualifier("retryConsumerFactory") ConsumerFactory<String, Object> consumerFactory,
    CommonErrorHandler commonErrorHandler) {

    ConcurrentKafkaListenerContainerFactory<String, Object> factory = new ConcurrentKafkaListenerContainerFactory<>();
    factory.setConsumerFactory(consumerFactory);
    factory.getContainerProperties().setAckMode(ContainerProperties.AckMode.RECORD);
    factory.setCommonErrorHandler(commonErrorHandler);
    return factory;
}
```

Blocking retry

Преимущества и недостатки

- Увеличиваем надежность системы
- Гарантируется доставки сообщений в том же порядке, что и пришли
- Весь остальной поток сообщений вынужден ждать
- Риск дубликатов

Non-Blocking retry

Преимущества и недостатки

- Увеличиваем надежность системы
- Весь остальной поток сообщений не блокируется
- Порядок сообщений не гарантируется
- Риск дубликатов
- Сложнее обрабатывать и поддерживать (так как используются отдельные топики)

Non-Blocking retry

Преимущества и недостатки

IMPORTANT

[Non-Blocking Retries](#) are not supported with batch listeners.

IMPORTANT

[Non-Blocking Retries](#) cannot combine with [Container Transactions](#). When the listener code throws an exception, container transaction commit succeeds, and the record is sent to the retryable topic.

Retry

Bottle neck

Group		Lag
script-executor-srv		1025050

Configs

RetryableTopic

```
• @RetryableTopic(  
    attempts = "${topic.telemetry.in.retry.attempts}",  
    exclude = {BusinessException.class, ValidationException.class, DeserializationException.class, NullPointerException.class, IllegalArgumentException.class},  
    kafkaTemplate = "retryKafkaTemplate",  
    listenerContainerFactory = "retryKafkaListenerContainerFactory",  
    • dltStrategy = DltStrategy.FAIL_ON_ERROR  
)  
@KafkaListener(topics = "${topic.telemetry.in}", containerFactory = "kafkaListenerContainerFactory", groupId = "topic.telemetry.in.group")  
public void consume(final TelemetryDto telemetryDto) {  
    •  
}
```

Configs

Consumer

@Bean

```
public ConsumerFactory<String, Object> retryConsumerFactory(KafkaProperties properties) {  
    final Map<String, Object> consumerProperties = properties.buildConsumerProperties();  
    consumerProperties.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);  
    consumerProperties.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, ByteArrayDeserializer.class);  
  
    return new DefaultKafkaConsumerFactory<>(consumerProperties);  
}
```

@Bean

```
public KafkaListenerContainerFactory<ConcurrentMessageListenerContainer<String, Object >> retryKafkaListenerContainerFactory(  
    @Qualifier("retryConsumerFactory") ConsumerFactory<String, Object> consumerFactory,  
    ObjectMapper objectMapper) {  
  
    ConcurrentKafkaListenerContainerFactory<String, Object> factory = new ConcurrentKafkaListenerContainerFactory<>();  
    factory.setConsumerFactory(consumerFactory);  
    factory.getContainerProperties().setAckMode(ContainerProperties.AckMode.RECORD);  
    factory.setMessageConverter(new StringJsonMessageConverter(objectMapper));  
    return factory;  
}
```

Configs

Producer

@Bean

```
public ProducerFactory<String, Object> retryProducerFactory(KafkaProperties properties) {  
    final Map<String, Object> producerProperties = properties.buildProducerProperties();  
    producerProperties.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);  
    producerProperties.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, ByteArraySerializer.class);  
  
    return new DefaultKafkaProducerFactory<>(producerProperties);  
}
```

@Bean

```
public KafkaTemplate<String, Object> retryKafkaTemplate(@Qualifier("retryProducerFactory")  
                                                         ProducerFactory<String, Object> retryProducerFactory,  
                                                         ObjectMapper objectMapper) {  
    final KafkaTemplate<String, Object> template = new KafkaTemplate<>(retryProducerFactory);  
    template.setMessageConverter(new StringJsonMessageConverter(objectMapper));  
    return template;  
}
```

Configs

RetryableTopic

```
@RetryableTopic(  
    attempts = "${topic.telemetry.in.retry.attempts}",  
    exclude = {BusinessException.class, ValidationException.class, DeserializationException.class, NullPointerException.class, IllegalArgumentException.class},  
    kafkaTemplate = "retryKafkaTemplate",  
    listenerContainerFactory = "retryKafkaListenerContainerFactory",  
    •    dlStrategy = DltStrategy.FAIL_ON_ERROR  
)  
@KafkaListener(topics = "${topic.telemetry.in}", containerFactory = "kafkaListenerContainerFactory", groupId = "topic.telemetry.in.group")  
public void consume(final TelemetryDto telemetryDto) {  
    •  
}
```

DLT Strategies

- **FAIL_ON_ERROR** – отсутствие переотправки в случае сбоя
- **ALWAYS_RETRY_ON_ERROR** (**default**) - всегда происходит переотправка в случае сбоя
- ***NO_DLT*** - Не создает DLT / отключает его

Configs

DltHandler

- Для обработки сообщений из DLT есть отдельный `@DltHandler`
- Он должен быть помещен в тот же класс, что и метод с аннотацией `@KafkaListener`.

Configs

DltHandler

```
@DltHandler
public void handle(@NonNull Message<?> msg,
    @Header(KafkaHeaders.EXCEPTION_STACKTRACE) String stacktrace,
    @Header(KafkaHeaders.EXCEPTION_MESSAGE) String errorMessage,
    @Header(KafkaHeaders.RECEIVED_TOPIC) String topicName) {
    handleDlt(msg, stacktrace, errorMessage, topicName);
}
```

```
public static void handleDlt(Message<?> msg, String stacktrace, String errorMessage, String topicName) {
    Object payload = (msg.getPayload() instanceof byte[] bytes)
        ? new String(bytes, StandardCharsets.UTF_8)
        : msg.getPayload();

    log.info("Received message from topic: '{}'. Error: {}. Message body: {}", topicName, errorMessage, payload);
    log.info("Message processing stacktrace: {}", stacktrace);
}
```

Configs

RetryableTopic

```
@RetryableTopic(  
    backoff = @Backoff(  
        delayExpression = "${topic.telemetry.in.retry.backoff.delay}",  
        multiplierExpression = "${topic.telemetry.in.retry.backoff.multiplier}",  
        maxDelayExpression = "${topic.telemetry.in.retry.backoff.max-delay}"),  
    •  
    attempts = "${topic.telemetry.in.retry.attempts}",  
    exclude = {BusinessException.class, ValidationException.class, DeserializationException.class, NullPointerException.class, IllegalArgumentException.class},  
    dltStrategy = DltStrategy.FAIL_ON_ERROR,  
    kafkaTemplate = "retryKafkaTemplate",  
    listenerContainerFactory = "retryKafkaListenerContainerFactory"  
)  
@KafkaListener(topics = "${topic.telemetry.in}", containerFactory = "kafkaListenerContainerFactory", groupId = "topic.telemetry.in.group")  
public void consume(final TelemetryDto telemetryDto) {  
    •  
}
```

DLT Topics

NETWORK.TOPIC.TELEMETRY.IN-dlt

NETWORK.TOPIC.TELEMETRY.IN-retry-1125

NETWORK.TOPIC.TELEMETRY.IN-retry-1687

NETWORK.TOPIC.TELEMETRY.IN-retry-500

NETWORK.TOPIC.TELEMETRY.IN-retry-750

DLT

Combining Blocking/Non-Blocking Retries

→ Используется сначала blocking, потом non-blocking

→ Полезно в случае микро задержек

DLT

Combining Blocking/Non-Blocking Retries

- `@Configuration`
- `public class` CombiningBlockNonBlock `extends` RetryTopicConfigurationSupport {

 `@Override`
 `protected void configureBlockingRetries`(BlockingRetriesConfigurer blockingRetries) {
 blockingRetries
 .retryOn(MyBlockingRetryException.class, MyOtherBlockingRetryException.class)
 .backOff(new FixedBackOff(interval, maxAttempts));
 }
}

Retry Producer

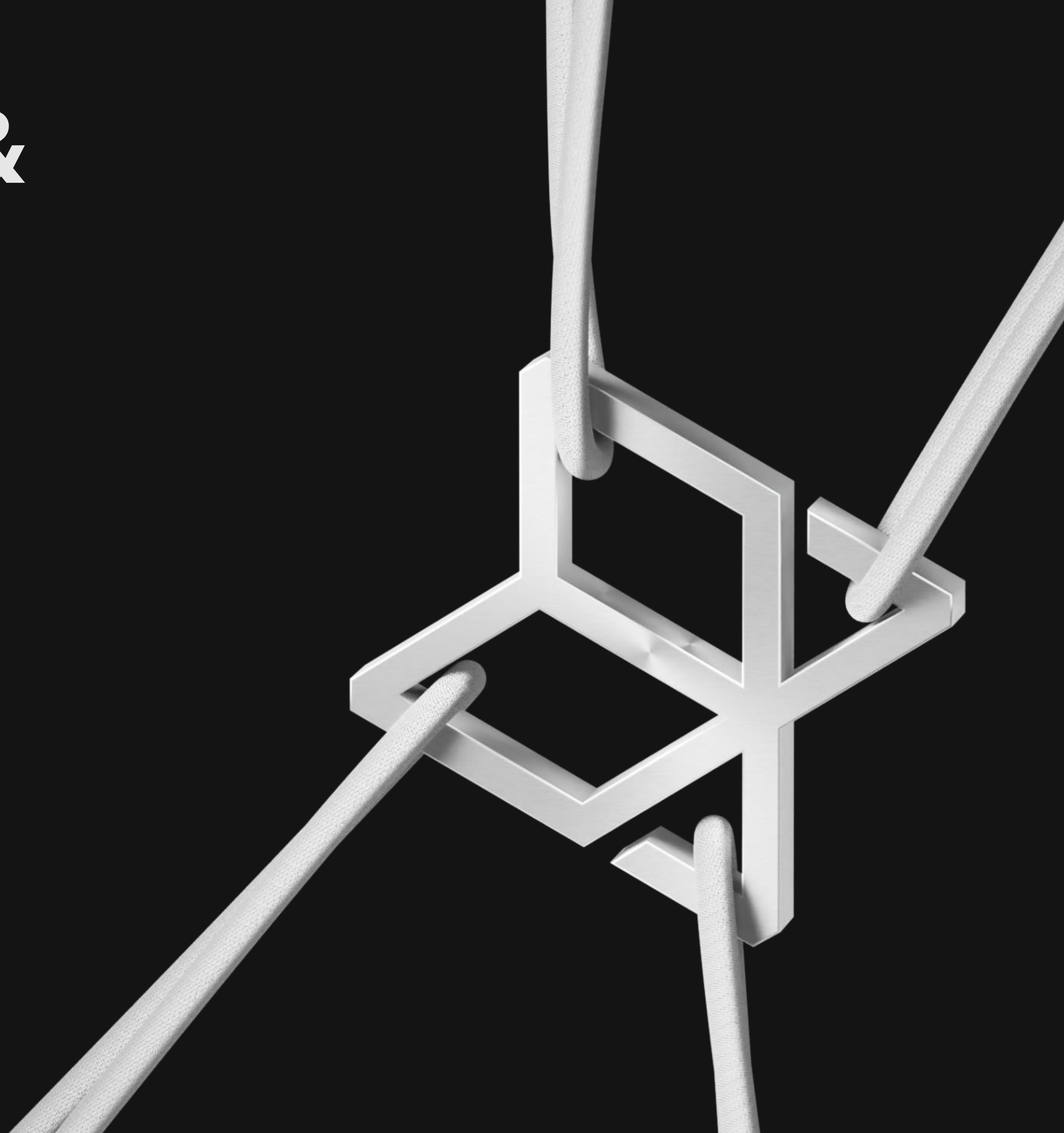
- Есть нативный retry механизм от самой kafka-client библиотеки (не Spring)
- Настраивается конфигами у producer'a

Retry Producer

@Bean

```
public ProducerFactory<String, Object> producerFactory() {  
    final Map<String, Object> producerProperties = new HashMap<>();  
  
    producerProperties.put(ProducerConfig.RETRIES_CONFIG, 20); // default 0  
    producerProperties.put(ProducerConfig.RETRY_BACKOFF_MS_CONFIG, "100");  
    producerProperties.put(ProducerConfig.RETRY_BACKOFF_MAX_MS_CONFIG, "1000");  
    producerProperties.put(ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, "5000");  
  
    return new DefaultKafkaProducerFactory<>(producerProperties);  
}
```

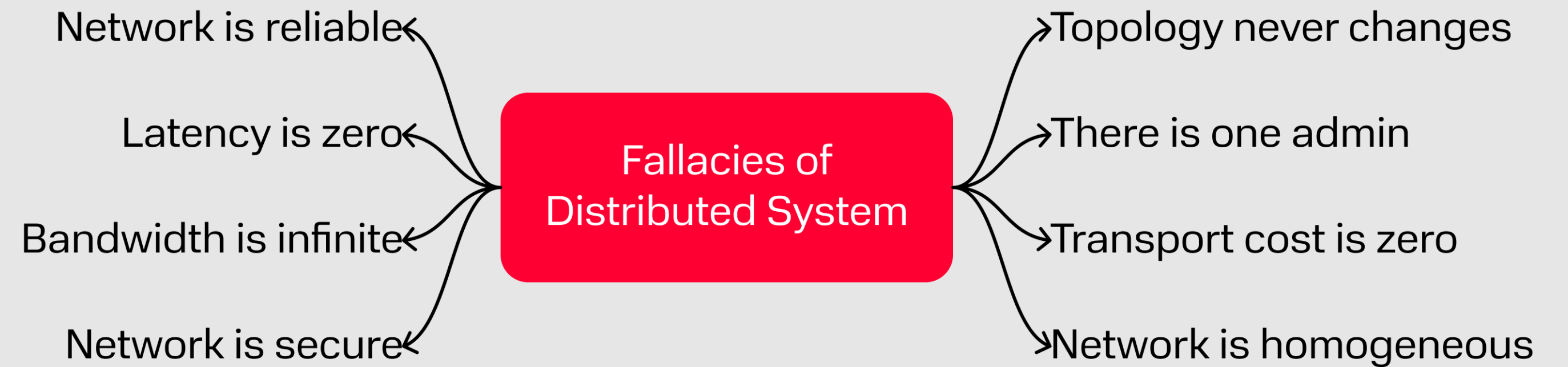
Kafka semantics & transaction



Durability

Проблемы которые могут возникнуть в системе с сообщением:

- Сбой сервиса при отправке в кафку
- Сбой сети при отправке
- Сбой ноды брокера
- Сбой сети при отправке ответа
- Сбой сервиса при получении ответа



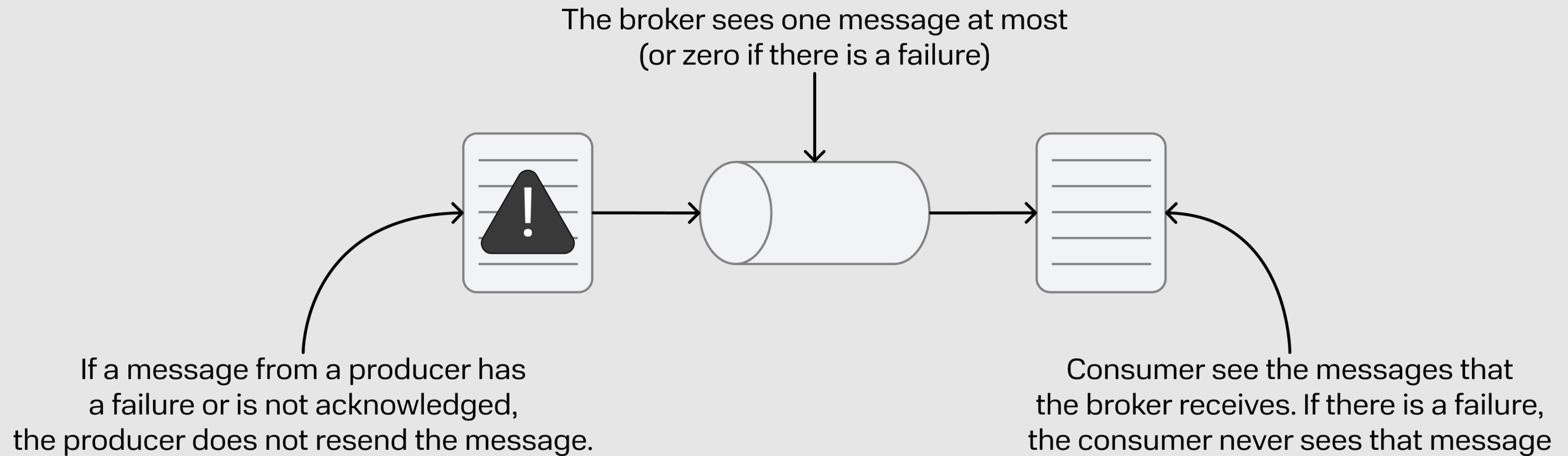
Kafka Semantics

(Гарантия доставки сообщений)

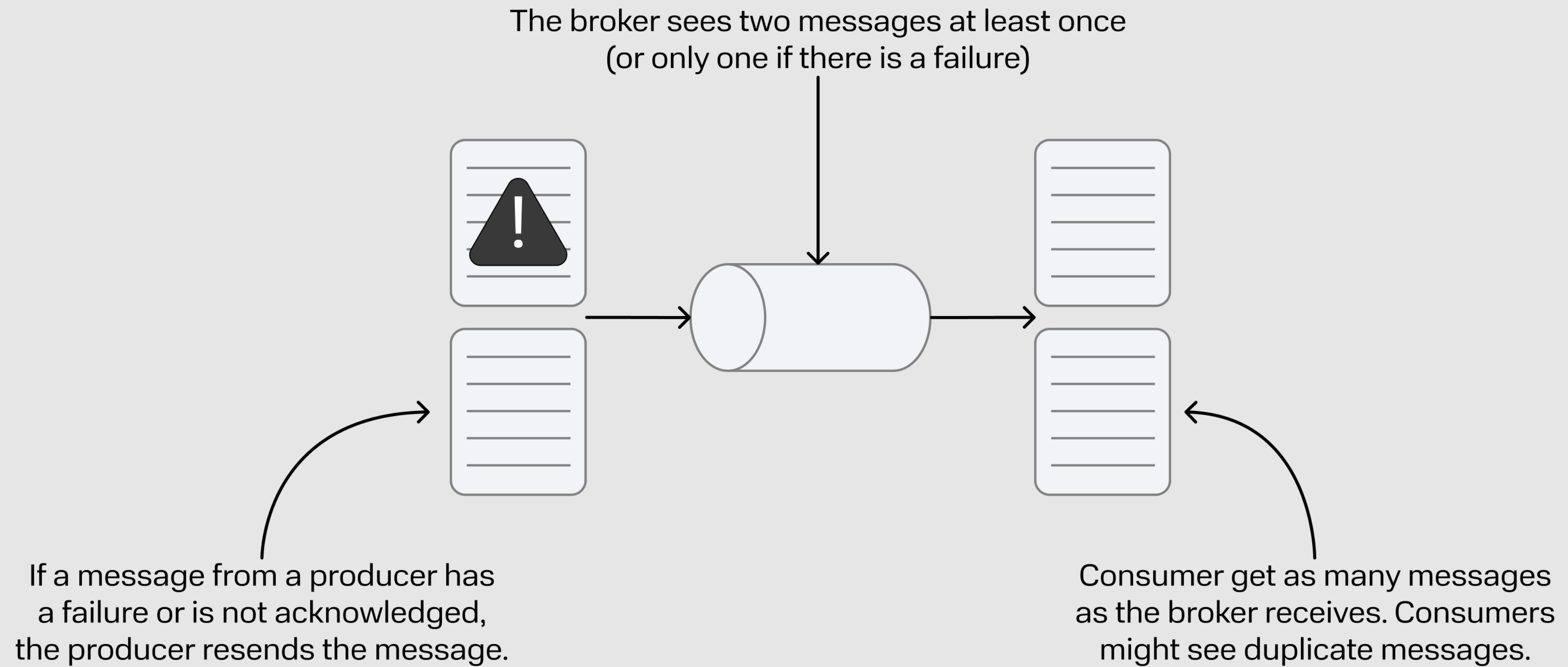
Существуют 3 вида семантики по доставке сообщения:

- At-most-once — сообщение отправится один раз и никак не обработается, если произошла ошибка
- At-least-once — сообщение отправляется столько раз, пока не произойдет подтверждение (acknowledge)
- Exactly-once — сообщение доставится ровно один раз

At-most-once



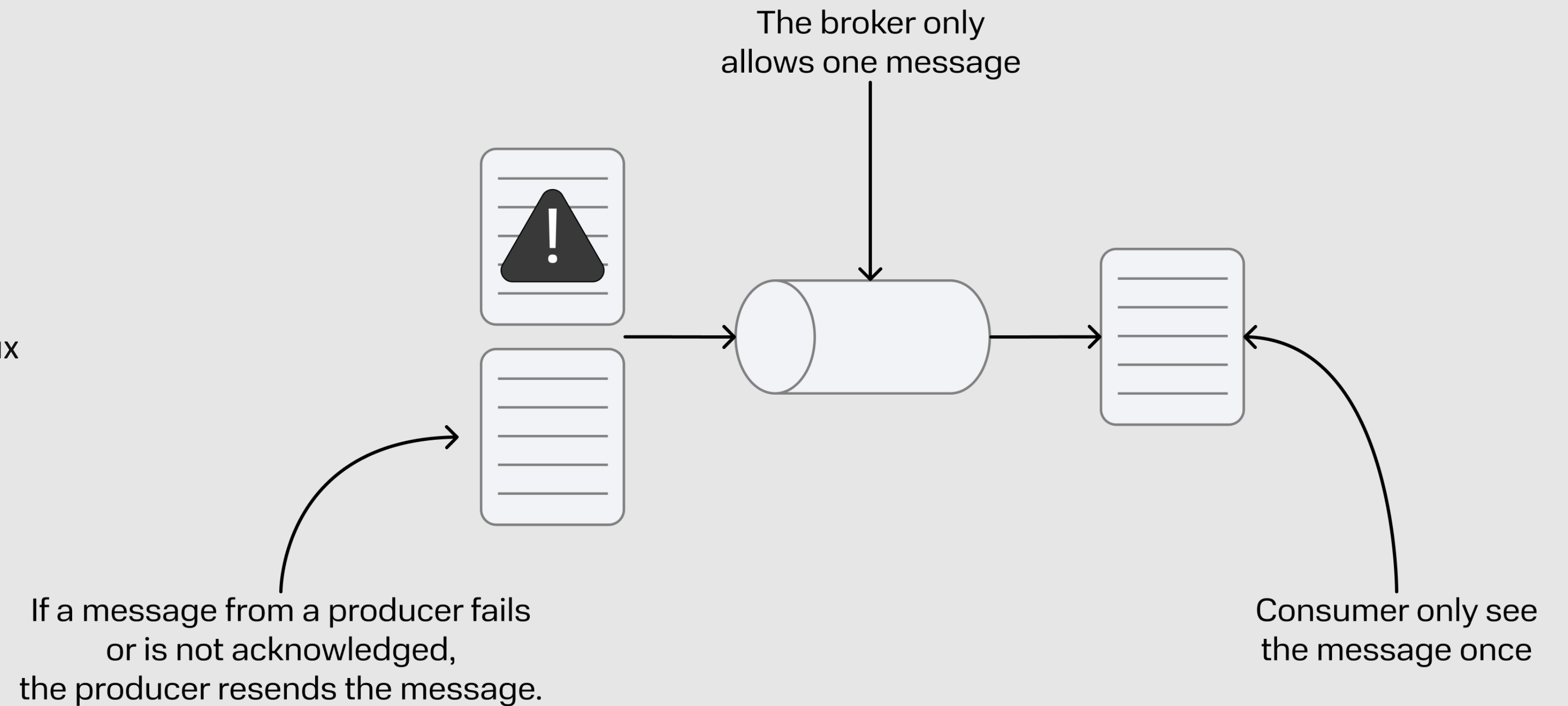
At-least-once



Exactly-once

Есть 3 основных способа
по обеспечению семантики EOS:

- Использование транзакционных producers и consumers.
- Использовать Kafka Streams.
- При Kafka Connect — использование поддерживаемых имплементаций sink/source для EOS



Producer delivery

- At-most-once
- At-least-once
- Idempotent producer
- Exactly-once

Producer delivery

At most once

- Нет повторной отправки.
- Возможна потеря сообщений.
- Минимальный оверхед на производительность и latency.

Producer delivery

At least once

- Возможны дубликаты сообщений (можно решить за счет добавления включения idempotent producer).
- Нет гарантии порядка сообщений.
- Сами паттерны Retry + DLT не имеют гарантии что у вас данная семантика будет соблюдаться

Producer delivery

Idempotent producer

Включается параметром: “*enable.idempotence*”: “*true*”.

Producer delivery

Idempotent producer in Spring

Требования к Idempotent producer:

- Параметр `MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION` — меньше или равно значению `MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION_FOR_IDEMPOTENCE` (по умолчанию выставлено как 5).
- Параметр `RETRIES_CONFIG` — больше чем 0.
- Параметр `ACKS_CONFIG` — выставлен как «all».

Прочее по Idempotent producer:

- Включается параметром: `(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true)`
- Включен по умолчанию, если все значения параметров соответствуют.
- Если Idempotent producer включен явно и настройки не соответствуют требуемым, то произойдет `ConfigException`.

Producer delivery

Idempotent producer in Spring

`enable.idempotence`

```
public static final String ENABLE_IDEMPOTENCE_CONFIG = "enable.idempotence";
public static final String ENABLE_IDEMPOTENCE_DOC = "When set to 'true', the producer will ensure that exactly one copy of each message is delivered to the consumer. If a message is retried due to broker failures, etc., may write duplicates of the retried message in the stream. "
    + "Note that enabling idempotence requires <code>" + MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION + "</code> to be less than or equal to the number of partitions in the topic (with message ordering preserved for any allowable value), <code>" + RETRIES_CONFIG + "</code> to be greater than 0, and <code>" + ACKS_CONFIG + "</code> must be 'all'. "
    + "<p>"
    + "Idempotence is enabled by default if no conflicting configurations are set. "
    + "If conflicting configurations are set and idempotence is not explicitly enabled, idempotence is disabled. "
    + "If idempotence is explicitly enabled and conflicting configurations are set, a <code>ConfigException</code> is thrown.";
```

Producer delivery Exactly once

- Достигается за счет транзакции (двухфазного коммита)
- Решает все проблемы с дубликатами и потерей сообщений
- Решает проблемы влияния GC и прочих непредвиденных инцидентов на доставку сообщений
- Возможна проблема с latency
- Влияние на производительность — минимальны (по заявлениям от Confluent)
- Транзакции требуют в кластере по крайней мере 3 брокера (это число можно поменять изменив параметр `transaction.state.log.replication.factor`)

Producer delivery Transactions

```
props.put("enable.idempotence", true);  
props.put("transactional.id", "prod-1");  
props.put("transaction.timeout.ms", 60000); // default
```

Producer delivery Transactions

```
producerFactory.createProducer(this.transactionIdPrefix);
```

```
producer.beginTransaction();
```

```
// produce message, do something
```

```
producer.commitTransaction();
```

```
producer.abortTransaction();
```

```
closeProducer(producer);
```

Producer delivery Transactions in Spring

- `kafkaTemplate.executeInTransaction(...);`

или

```
@Transactional("kafkaTransactionManager")  
public void sendToKafka(SomeDto dto) {  
    kafkaTemplate.send("topic", dto);  
}
```

Consumer receiving

→ At-most-once

→ At-least-once

→ Exactly-once

→ Без семантики (практический случай)

Consumer receiving At most once

```
@KafkaListener(topics = "#{ '${spring.kafka.incoming.topic}' }")  
public void listen(ConsumerRecord<String, Object> messages, Acknowledgment acknowledgment) {  
    acknowledgment.acknowledge();  
    ...  
}
```

Consumer receiving At least once

```
@KafkaListener(topics = "#{ '${spring.kafka.incoming.topic}' }")
public void listen(ConsumerRecord<String, Object> messages, Acknowledgment acknowledgment) {
    ...
    acknowledgment.acknowledge();
}
```

Consumer receiving Exactly once

Isolations.level:

- `read_uncommitted` — чтение всех сообщений (грязное чтение) // default
- `read_committed` — чтение только закомиченных сообщений

Практический опыт Без семантики

На практике может получиться так, что данные на входе получили одним из возможным способом и отправляем их далее через `async producer`

- Сообщение может доставится 0 раз
- Сообщение может задублироваться
- Семантика от 0 сообщений до N (Unknown once)

Transactions

Consumer + Producer

- KafkaTransactionManager
- DataSourceTransactionManager
- @Transactional
- Chained kafka + db transaction:
 - KafkaTransactionManager c @KafkaListener
 - DataSourceTransactionManager c @Transactional

Transactions

Consumer + Producer

```
producerFactory.createProducer(this.transactionIdPrefix);

producer.beginTransaction();
// produce message, do something
producer.sendOffsetsToTransaction(consumedOffsets, consumerGroupId);

producer.commitTransaction();
producer.abortTransaction();

closeProducer(producer);
```

Transactions

Consumer + Producer

```
@Transactional("kafkaTransactionManager")
@KafkaListener(topics = "#{ '${spring.kafka.incoming.topic}' }")
public void listen(ConsumerRecord<String, Object> messages) {
    ...
}
```

Transaction: Kafka + DB

- Kafka не поддерживает глобальные, гетерогенные транзакции (XA transactions)
- Атомарности достичь невозможно
- Возможно лишь уменьшить временное окно, между 2-мя коммитами и достичь определенной отказоустойчивости, но без 100% гарантии

Transaction: Kafka + DB + Spring

- Используется @Transactional
 - Используется retry
 - Операции в бд должны быть идемпотентными
 - Вся реализация довольно хрупкая и не является надежной
-
- Not rollback transactions in database with KafkaTemplate and kafka down
 - Offset is committed after the rollback of the DB transaction
 - Should nested transactions synchronize with Spring Kafka?

github.com/spring-projects/spring-kafka/issues/2411

github.com/spring-projects/spring-kafka/issues/2715

docs.spring.io/spring-kafka/reference/tips.html#ex-jdbc-sync

Transaction:

Kafka + DB - коммитится первой транзакция в бд

```
@Transactional("DataSourceTransactionManager")
public void process(SomeEntity entity, SomeDto dto) {
    kafkaTemplate.send("topic", dto);
    saveToDb(entity);
}
```

Transaction:

Kafka + DB - коммится первой транзакция в кафке

```
@Transactional("DataSourceTransactionManager")
public void someMethod(SomeEntity entity, SomeDto dto) {
    saveToDb(entity);
    sendToKafka(in);
}
```

```
@Transactional("kafkaTransactionManager")
public void sendToKafka(SomeDto dto) {
    kafkaTemplate.send("topic", dto));
}
```

Kafka Connect Exactly once

- Sink connector
- Source connector

Kafka Connect Exactly once

Sink connector:

- Нужно смотреть поддержку EOS в каждой реализации определенного коннектора отдельно
- Нужно выставить на воркере:
`consumer.isolation.level: read_committed`
- Или выставить в override policy:
`consumer.override.isolation.level: read_committed`
- Требования по ACL отсутствуют (access control lists)

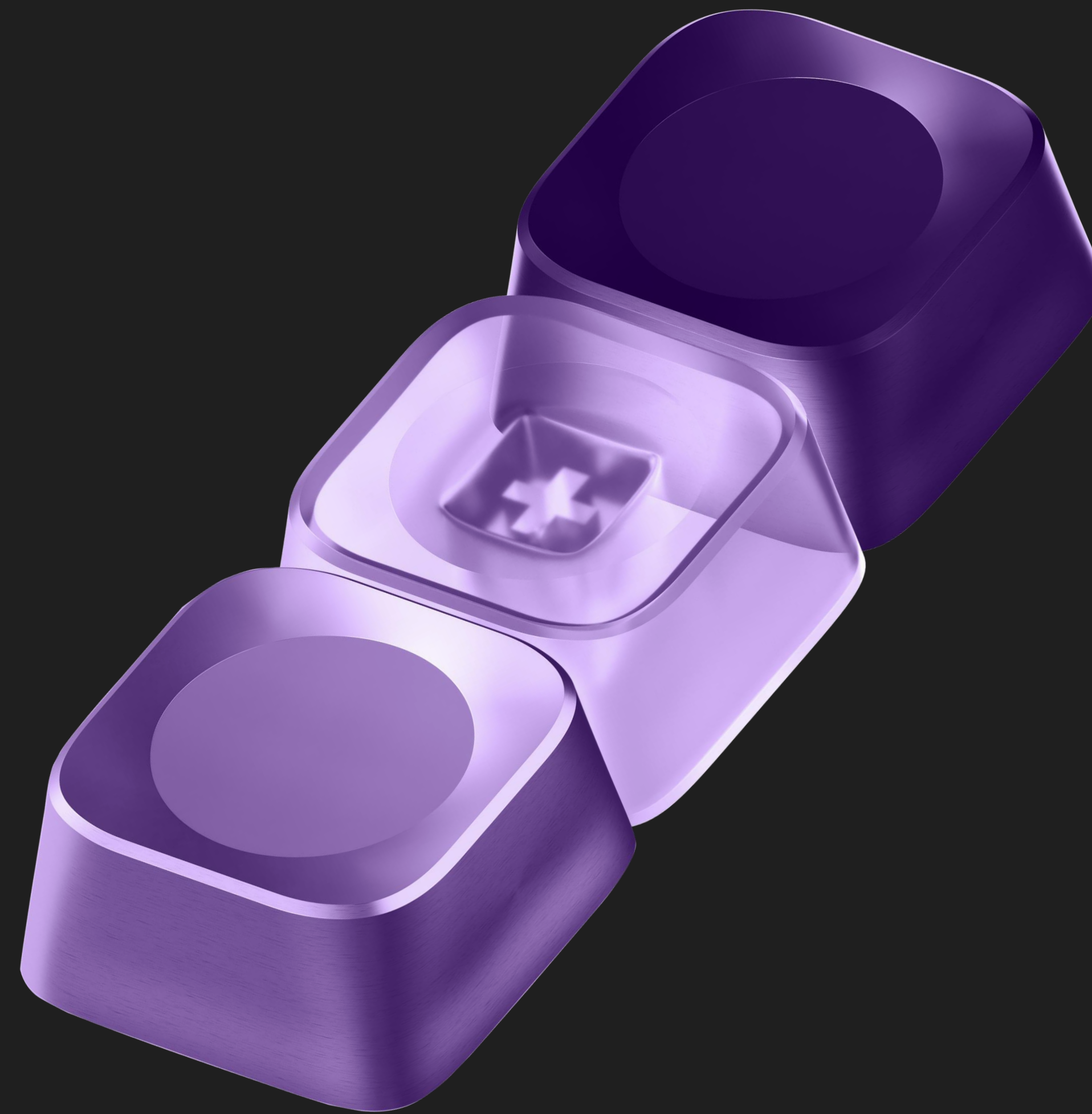
Kafka Connect Exactly once

Source connector:

- Поддерживается с версии 3.3.0
- Но нужно смотреть поддержку EOS в каждой реализации определенного коннектора отдельно
- Поддерживаются только коннекторы работающие по distributed mode;
- standalone connect workers семантику EOS не поддерживают
- ACL необходим, если развернут в защищенном кафка кластере

- Для нового Kafka connect необходимо добавить параметр `exactly.once.source.support: enabled`
- Для существующего необходимо конфиг `exactly.once.source.support:`
 1. Сначала перевести в состояние `preparing`
 2. Затем уже в `enabled`

Вопросы



Минутка обратной связи

